

Space Complexity of Reachability in Simple Path Graphs

Krishnamoorthy Dinesh and **Chandana Sasidharan**

Department of Computer Science and Engineering
IIT Palakkad, India

August 25, 2026

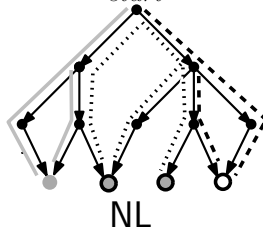
Introduction

Configuration Graph of a TM on input x

start



start



Accept



Reject



Abort

Introduction

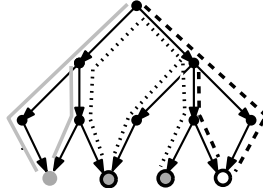
Configuration Graph of a TM on input x

start



L

start



NL



Accept



Reject



Abort

$L = \text{DSPACE}(\log n)$

$NL = \text{NSPACE}(\log n)$

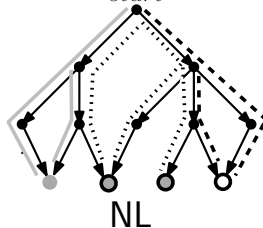
Introduction

Configuration Graph of a space bounded TM

start



start



Accept



Reject



Abort

- $L \subseteq NL$.

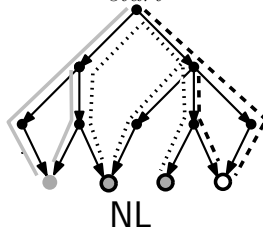
Introduction

Configuration Graph of a space bounded TM

start



start



Accept



Reject



Abort

- $L \subseteq NL$. Is $L = NL$?

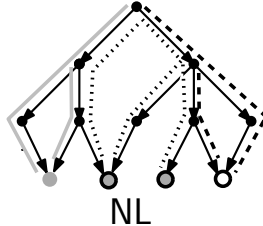
Introduction

Configuration Graph of a space bounded TM

start



start



Accept



Reject



Abort

- $L \subseteq NL$. Is $L = NL$?
- Best known : $NL \subseteq L^2$ (Savitch's Theorem)

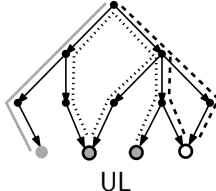
A class between NL and L

Unambiguous Logspace

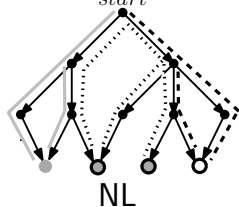
start



start



start



Accept



Reject



Abort

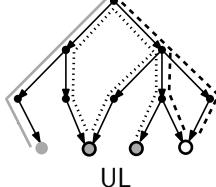
A class between NL and L

Unambiguous Logspace

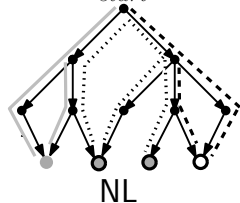
start



start



start



Accept



Reject



Abort

- **UL**: At most one accepting path.

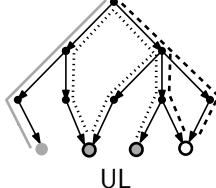
A class between NL and L

Unambiguous Logspace

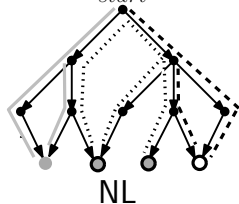
start



start



start

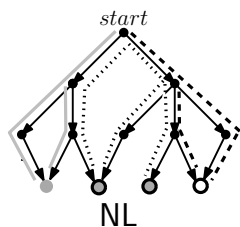
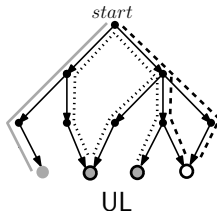
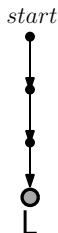


● Accept ● Reject ○ Abort

- **UL**: At most one accepting path. Abort & Reject: No constraints.

A class between NL and L

Unambiguous Logspace

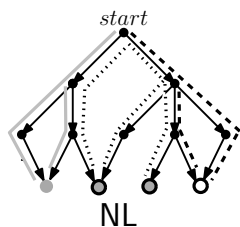
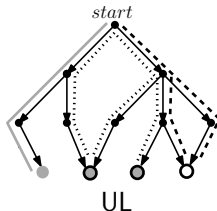
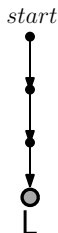


● Accept ● Reject ○ Abort

- UL: At most one accepting path. Abort & Reject: No constraints.
- What if there are two accepting paths?

A class between NL and L

Unambiguous Logspace

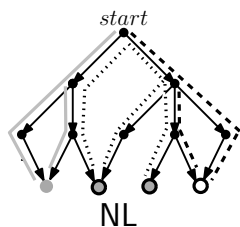
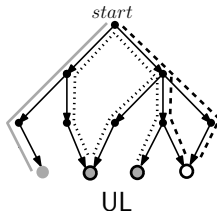


● Accept ● Reject ○ Abort

- UL: At most one accepting path. Abort & Reject: No constraints.
- What if there are two accepting paths? Can never happen.

A class between NL and L

Unambiguous Logspace

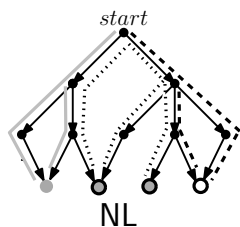
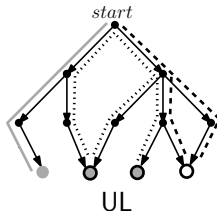


● Accept ● Reject ○ Abort

- UL: At most one accepting path. Abort & Reject: No constraints.
- What if there are two accepting paths? Can never happen.
- coUL: At most one rejecting path.

A class between NL and L

Unambiguous Logspace

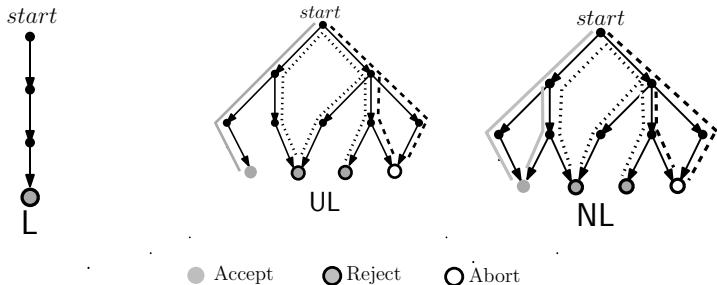


● Accept ● Reject ○ Abort

- UL: At most one accepting path. Abort & Reject: No constraints.
- What if there are two accepting paths? Can never happen.
- coUL: At most one rejecting path. Abort & Accept: No constraints.

A class between NL and L

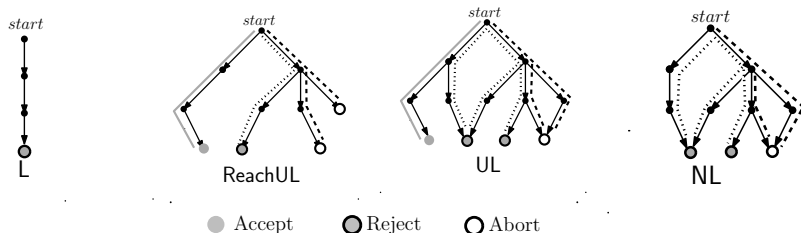
Unambiguous Logspace



- UL: At most one accepting path. Abort & Reject: No constraints.
- What if there are two accepting paths? Can never happen.
- coUL: At most one rejecting path. Abort & Accept: No constraints.

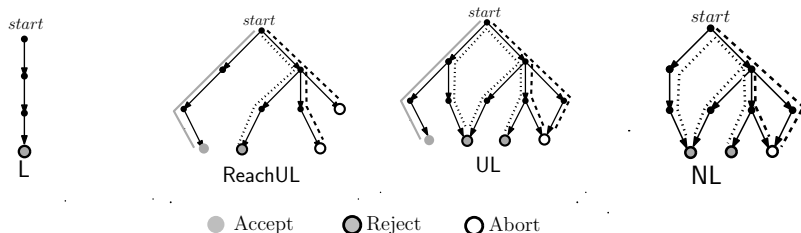
$$L \subseteq UL \cap \text{coUL} \subseteq UL \subseteq NL$$

More classes between NL and L



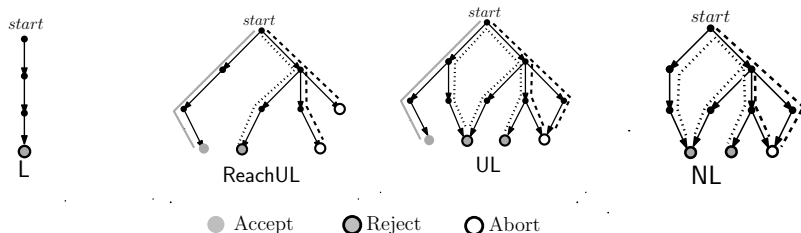
- **UL:** At most one accepting path. Abort & Reject: No constraints. Unique accept.
- **coUL:** At most one rejecting path. Abort & Accept: No constraints. Unique reject.

More classes between NL and L



- **UL**: At most one accepting path. Abort & Reject: No constraints. Unique accept.
- **coUL**: At most one rejecting path. Abort & Accept: No constraints. Unique reject.
- **ReachUL** (Buntrock *et al.*, FCT 1991): At most one path to every node. Unique accept and reject.

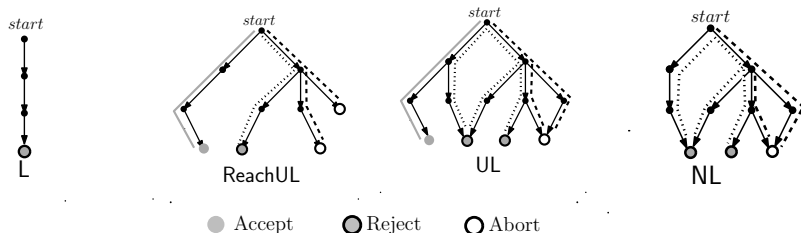
More classes between NL and L



- **UL**: At most one accepting path. Abort & Reject: No constraints. Unique accept.
- **coUL**: At most one rejecting path. Abort & Accept: No constraints. Unique reject.
- **ReachUL** (Buntrock *et al.*, FCT 1991): At most one path to every node. Unique accept and reject.

$$L \subseteq \text{ReachUL} \subseteq \text{UL} \cap \text{coUL} \subseteq \text{UL} \subseteq \text{NL}$$

More classes between NL and L



- UL: At most one accepting path. Abort & Reject: No constraints. Unique accept.
- coUL: At most one rejecting path. Abort & Accept: No constraints. Unique reject.
- ReachUL (Buntrock *et al.*, FCT 1991): At most one path to every node. Unique accept and reject.

$$L \subseteq \text{ReachUL} \subseteq \text{UL} \cap \text{coUL} \subseteq \text{UL} \subseteq \text{NL} \subseteq \text{P}.$$

Reachability in Restricted Graph Class

Reachability Restricted to $\mathcal{G}$

$$\mathcal{G}\text{-Reach} = \{(G, s, t) \mid \underbrace{G \in \mathcal{G}}_{\text{Recognition}} \text{ and there is an } \underbrace{s \text{ to } t \text{ path.}}_{\text{Reachability}}\}$$

Reachability in Restricted Graph Class

Reachability Restricted to $\mathcal{G}$

$$\mathcal{G}\text{-Reach} = \{(G, s, t) \mid \underbrace{G \in \mathcal{G}}_{\text{Recognition}} \text{ and there is an } \underbrace{s \text{ to } t \text{ path.}}_{\text{Reachability}}\}$$

Ex: Tree-Reach = $\{(G, s, t) \mid G \text{ is a tree and there is an } s \text{ to } t \text{ path.}\}$

Complete Problems for Each Class

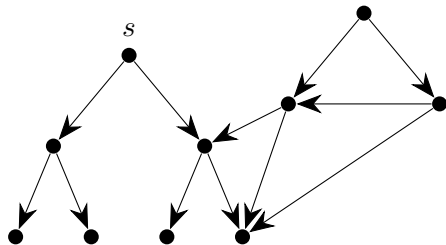
Complexity Class	L	UL	NL
Complete Problem	Undirected Reachability	No known complete problem	Directed Reachability

Complete Problems for Each Class

Complexity Class	L	UL	NL
Complete Problem	Undirected Reachability	No known complete problem	Directed Reachability

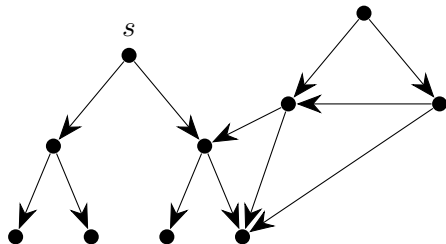
Does reach unambiguous class (ReachUL) have a complete problem?

Reach Unambiguous Graphs

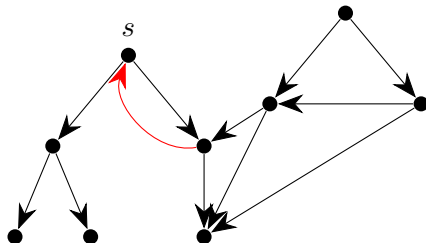


At most one path from s to any other vertex.

Reach Unambiguous Graphs

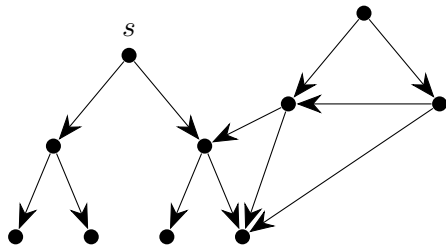


At most one path from s to any other vertex.

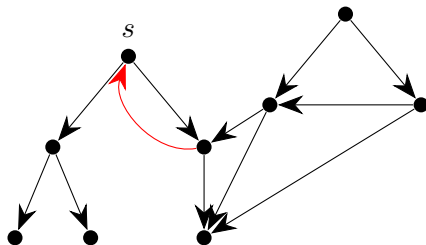


Non Example.

Reach Unambiguous Graphs



At most one path from s to any other vertex.



Non Example.

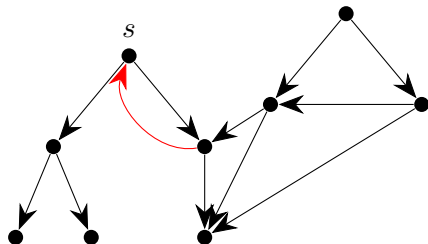
$$\text{RU-Reach} = \{G \text{ is reach unambiguous and there is } s \text{ to } t \text{ path.}\}$$

Complete Problems for Classes Seen So Far

Complexity Class	L	ReachUL	UL	NL
Complete Problem	Undirected Reachability	RU-Reach ¹	No known complete problem	Directed Reachability

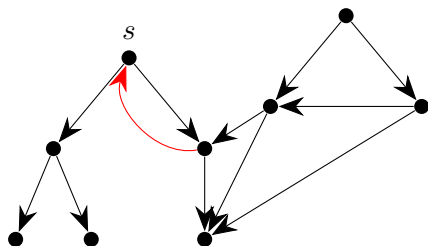
¹(Lange, STACS 97)

Example and Non-examples of Simple path graph



Example of Simple Path Graph

Example and Non-examples of Simple path graph

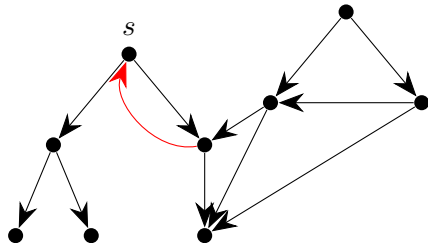


Example of Simple Path Graph

At most one **simple** path from s to any other vertex.

Complexity of Reachability

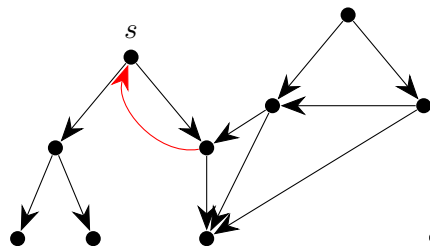
For Reach unambiguous graphs and Simple path graphs



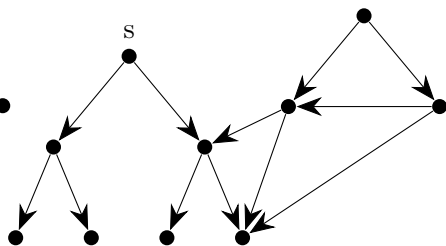
Simple Path Graph

Complexity of Reachability

For Reach unambiguous graphs and Simple path graphs



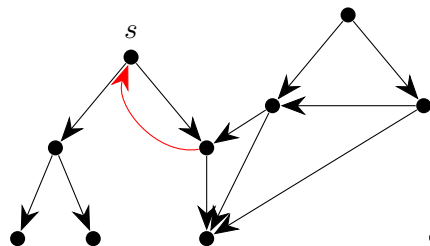
Simple Path Graph



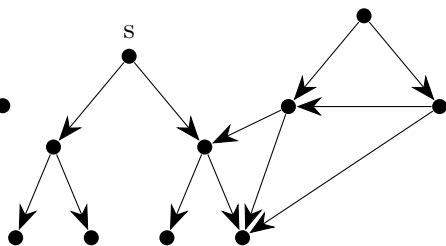
Reach Unambiguous Graph

Complexity of Reachability

For Reach unambiguous graphs and Simple path graphs



Simple Path Graph

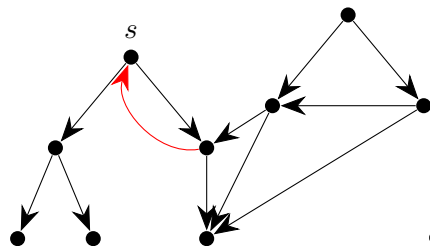


Reach Unambiguous Graph

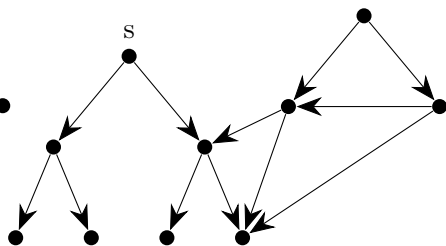
- Simultaneous
 - Time: $poly(n)$

Complexity of Reachability

For Reach unambiguous graphs and Simple path graphs



Simple Path Graph

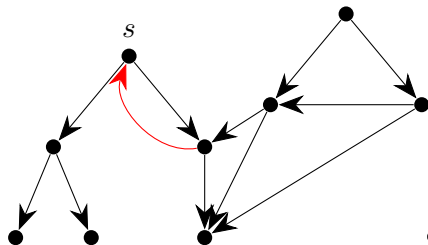


Reach Unambiguous Graph

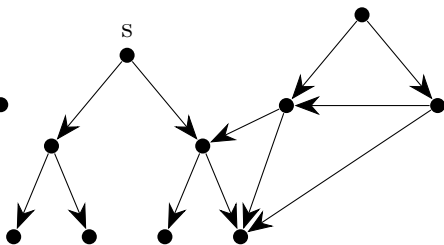
- Simultaneous
 - Time: $poly(n)$
 - Space:
 $O(n^{0.5}poly(\log n))$
- (Kannan *et al.*, FSTTCS 2008)

Complexity of Reachability

For Reach unambiguous graphs and Simple path graphs



Simple Path Graph



Reach Unambiguous Graph

- Simultaneous

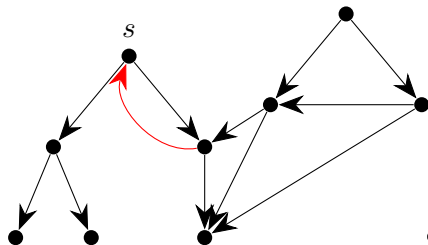
- Time: $\text{poly}(n)$
- Space:
 $O(n^{0.5} \text{poly}(\log n))$

- (Kannan *et al.*, FSTTCS 2008)

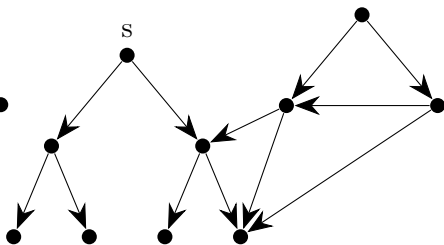
- Simultaneous

Complexity of Reachability

For Reach unambiguous graphs and Simple path graphs



Simple Path Graph



Reach Unambiguous Graph

- Simultaneous

- Time: $poly(n)$
- Space:
 $O(n^{0.5}poly(\log n))$

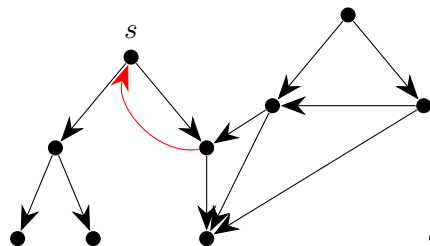
- (Kannan *et al.*, FSTTCS 2008)

- Simultaneous

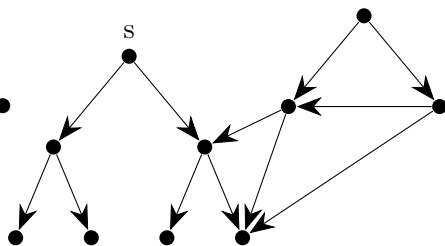
- Time: $poly(n)$

Complexity of Reachability

For Reach unambiguous graphs and Simple path graphs



Simple Path Graph



Reach Unambiguous Graph

- Simultaneous

- Time: $poly(n)$
- Space: $O(n^{0.5}poly(\log n))$

- (Kannan *et al.*, FSTTCS 2008)

- Simultaneous

- Time: $poly(n)$
- Space: $O(\log^2 n)$

- (Cook, STOC 1979)

Our Question and Result

SimplePath-Reach

$\{(G, s, t) \mid G \text{ is a simple path graph and there is an } s \text{ to } t \text{ path.}\}$

Our Question and Result

SimplePath-Reach

$\{(G, s, t) \mid G \text{ is a simple path graph and there is an } s \text{ to } t \text{ path.}\}$

Main Question

Is SimplePath-Reach $\in$ ReachUL?

Our Question and Result

SimplePath-Reach

$\{(G, s, t) \mid G \text{ is a simple path graph and there is an } s \text{ to } t \text{ path.}\}$

Main Question

Is SimplePath-Reach $\in$ ReachUL?

Consequence: Gives simultaneous $\text{poly}(n)$ time $O(\log^2 n)$ algo for SimplePath-Reach, improving the result of Kannan *et al.*

Our Question and Result

SimplePath-Reach

$\{(G, s, t) \mid G \text{ is a simple path graph and there is an } s \text{ to } t \text{ path.}\}$

Main Question

Is SimplePath-Reach $\in$ ReachUL?

Consequence: Gives simultaneous $\text{poly}(n)$ time $O(\log^2 n)$ algo for SimplePath-Reach, improving the result of Kannan *et al.*

We show

SimplePath-Reach $\in$ UL $\cap$ coUL.

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm**

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm (Only works for reach unambiguous graphs)**

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm (Only works for reach unambiguous graphs)**
- Can we extend to simple path graphs?

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm (Only works for reach unambiguous graphs)**
- Can we extend to simple path graphs?

Upcoming Slides:

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm (Only works for reach unambiguous graphs)**
- Can we extend to simple path graphs?

Upcoming Slides:

- Overview of Lange's algorithm.

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm (Only works for reach unambiguous graphs)**
- Can we extend to simple path graphs?

Upcoming Slides:

- Overview of Lange's algorithm.
- Challenges in adapting Lange's algorithm for reach unambiguous graphs.

Approach for Reachability in Simple Path Graphs

Modified Lange

- Many naive approaches such as BFS/DFS/Savitch does not work.
- Any other approach ?
- **Lange's Algorithm (Only works for reach unambiguous graphs)**
- Can we extend to simple path graphs?

Upcoming Slides:

- Overview of Lange's algorithm.
- Challenges in adapting Lange's algorithm for reach unambiguous graphs.
- How to overcome these challenges.

Lange's Algorithm Overview:

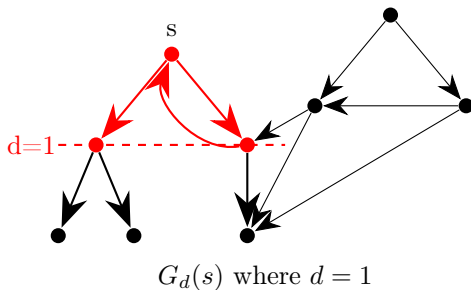
Each round, (start with $d = 1$),

- Pre-order traversal of $G_d(s)$, look for t .

Lange's Algorithm Overview:

Each round, (start with $d = 1$),

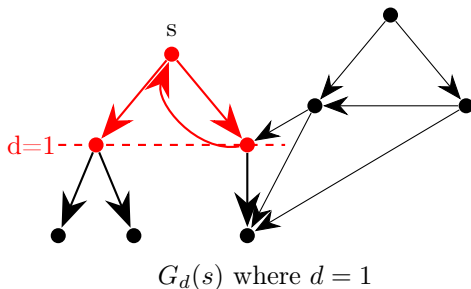
- Pre-order traversal of $G_d(s)$, look for t .



Lange's Algorithm Overview:

Each round, (start with $d = 1$),

- Pre-order traversal of $G_d(s)$, look for t .

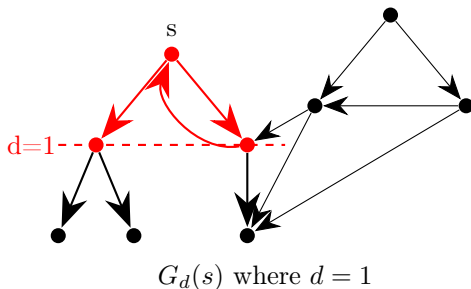


- Verify $G_d(s)$ is a reach unambiguous upto d .

Lange's Algorithm Overview:

Each round, (start with $d = 1$),

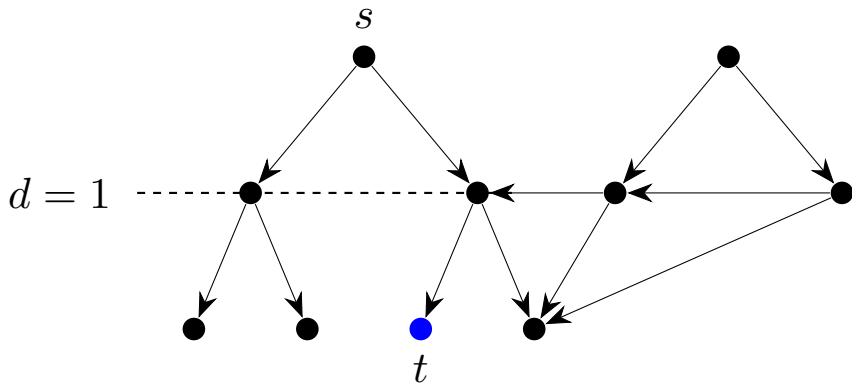
- Pre-order traversal of $G_d(s)$, look for t .



- Verify $G_d(s)$ is a reach unambiguous upto d .

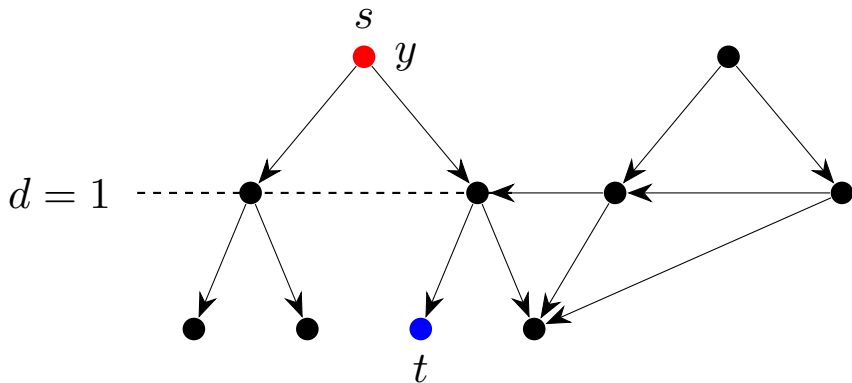
$\text{Next}(y, h)$: returns pre-order successor of y in $G_d(s)$ + its distance from s .

Lange's algorithm informally

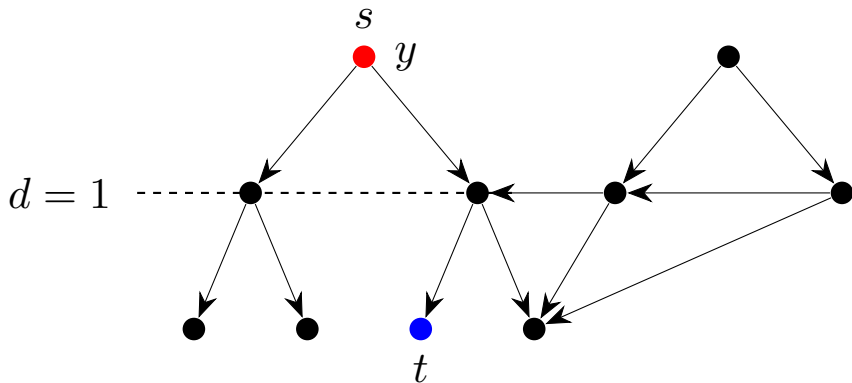


Pre-order traversal for y starts.

Lange's algorithm informally

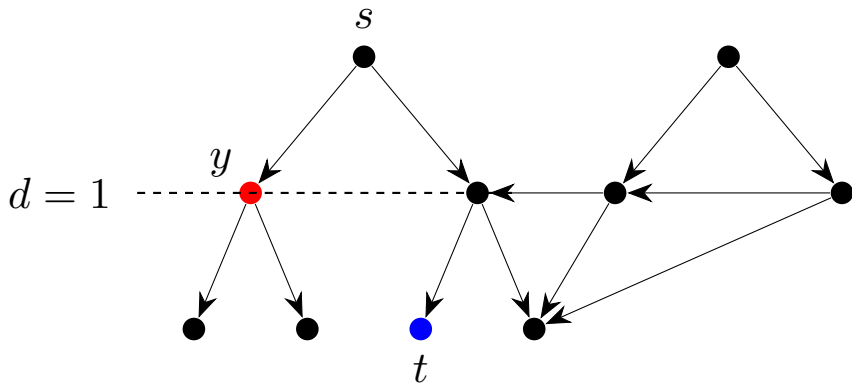


Lange's algorithm informally

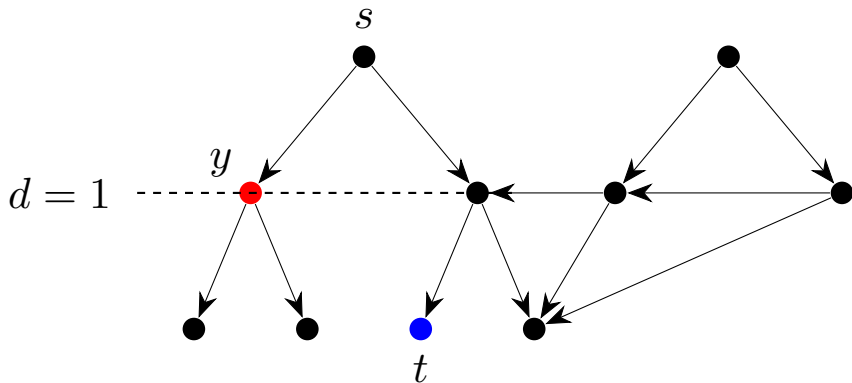


Call **Next**(y, h)

Lange's algorithm informally

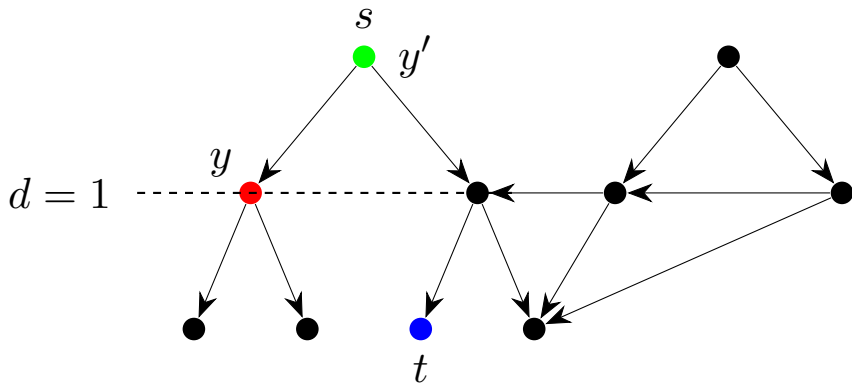


Lange's algorithm informally

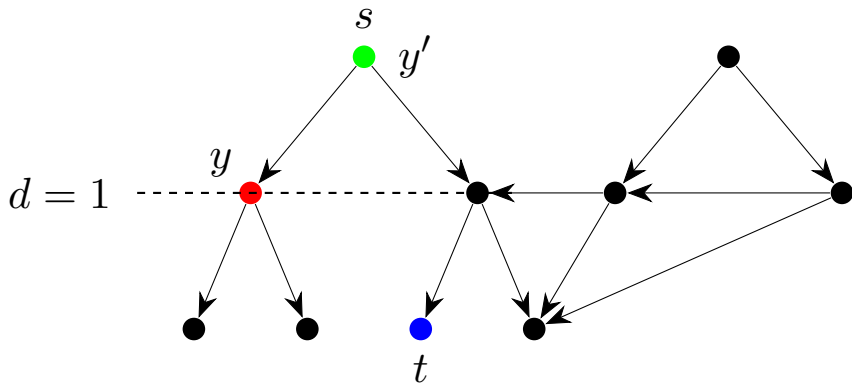


Pre-order traversal for y' starts

Lange's algorithm informally

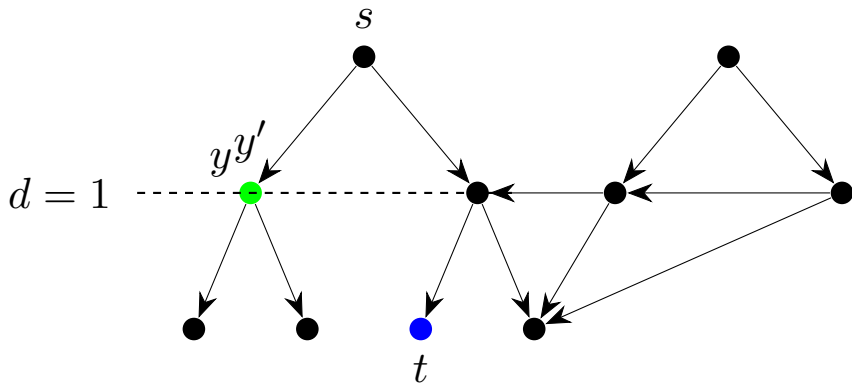


Lange's algorithm informally

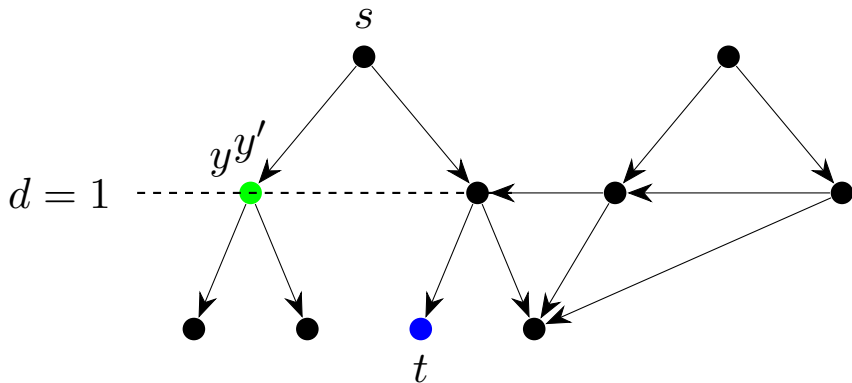


Call **Next**(y', h')

Lange's algorithm informally

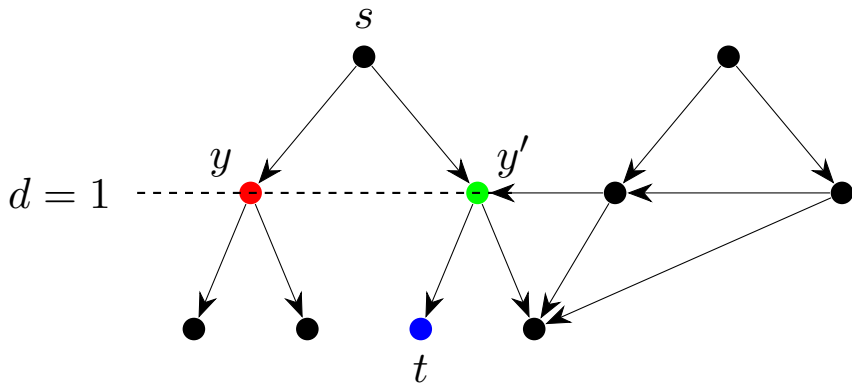


Lange's algorithm informally

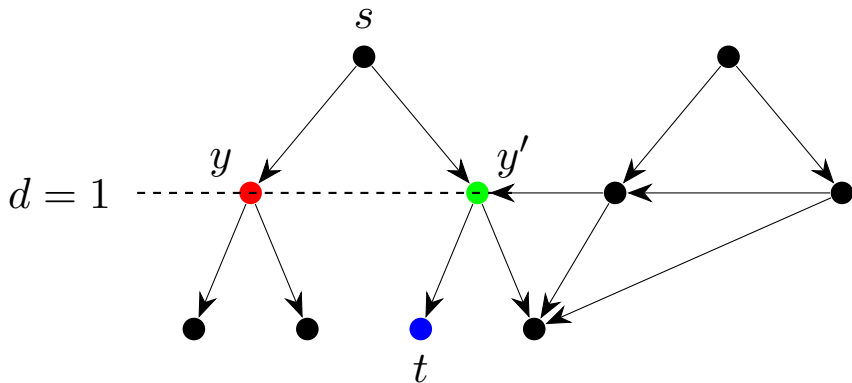


Call **Next**(y', h')

Lange's algorithm informally

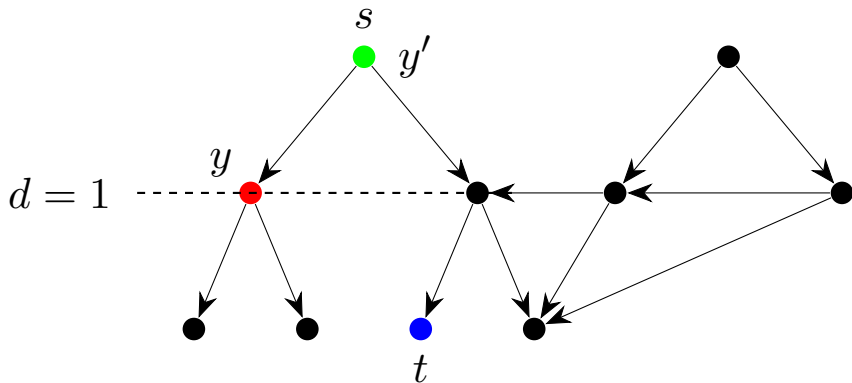


Lange's algorithm informally

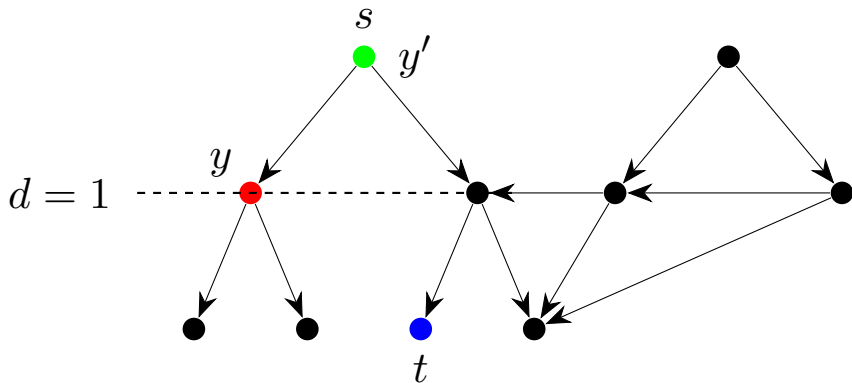


Call **Next**(y', h')

Lange's algorithm informally

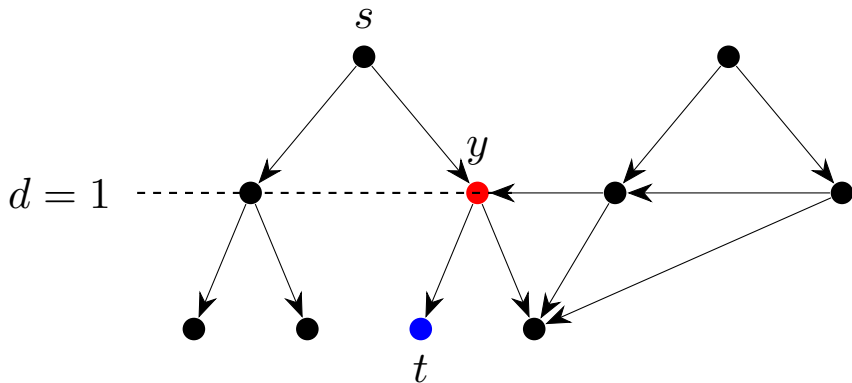


Lange's algorithm informally

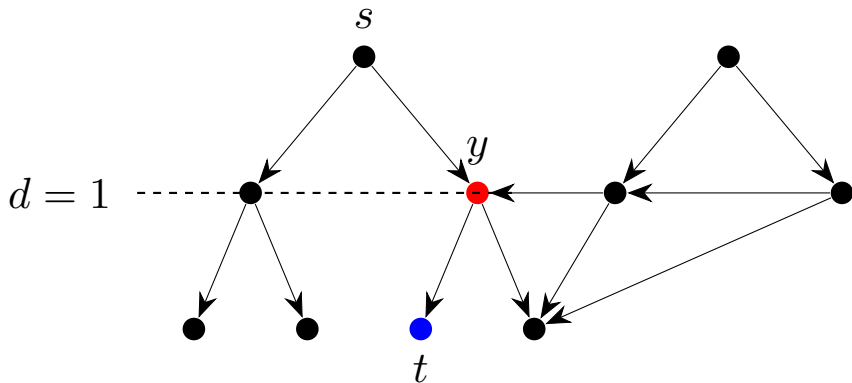


Call $\text{Next}(y, h)$

Lange's algorithm informally

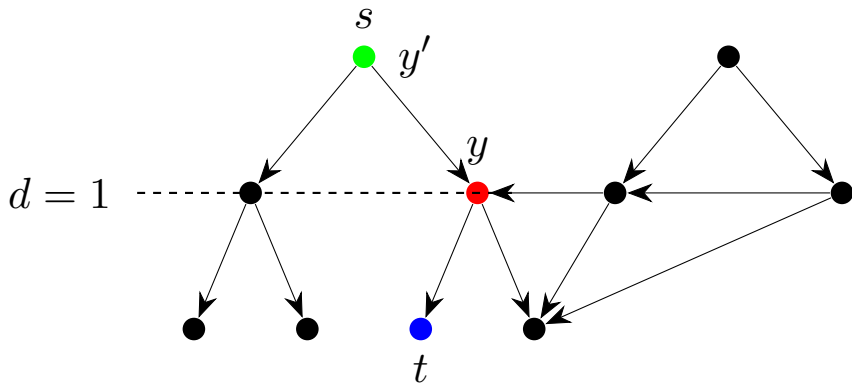


Lange's algorithm informally

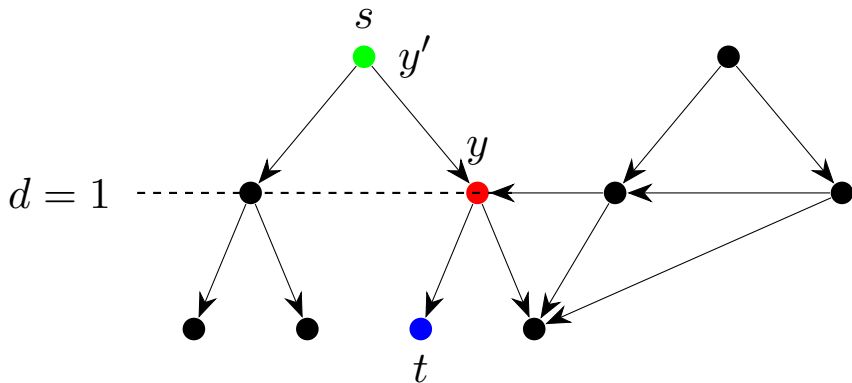


Pre-order traversal for y' starts

Lange's algorithm informally

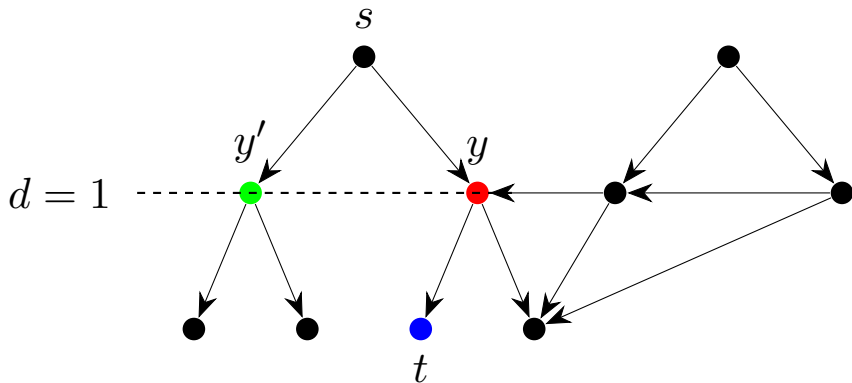


Lange's algorithm informally

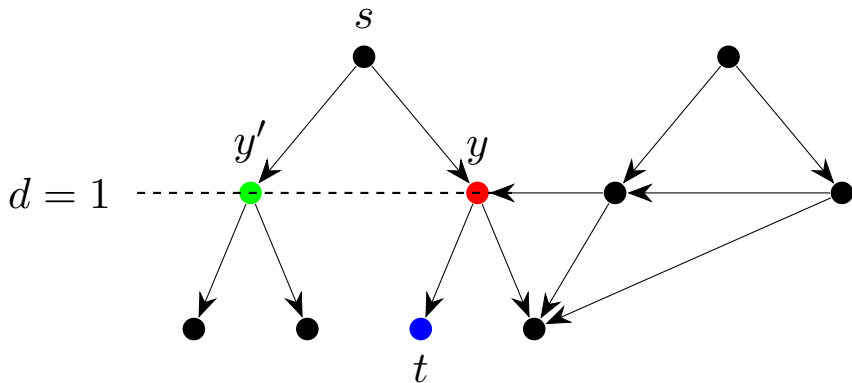


Call **Next**(y', h')

Lange's algorithm informally

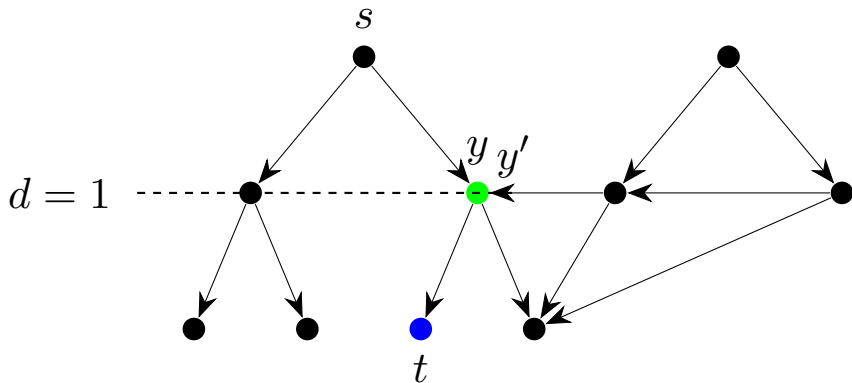


Lange's algorithm informally

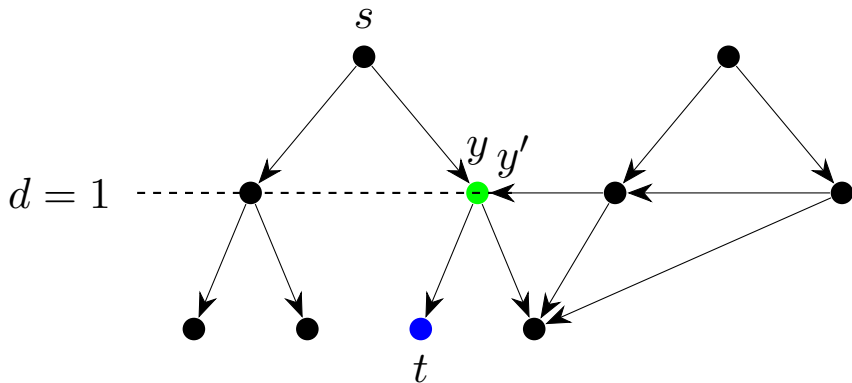


Call **Next**(y', h')

Lange's algorithm informally

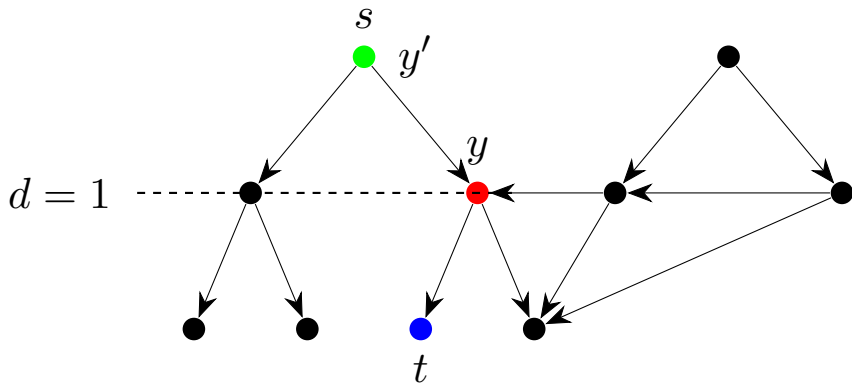


Lange's algorithm informally

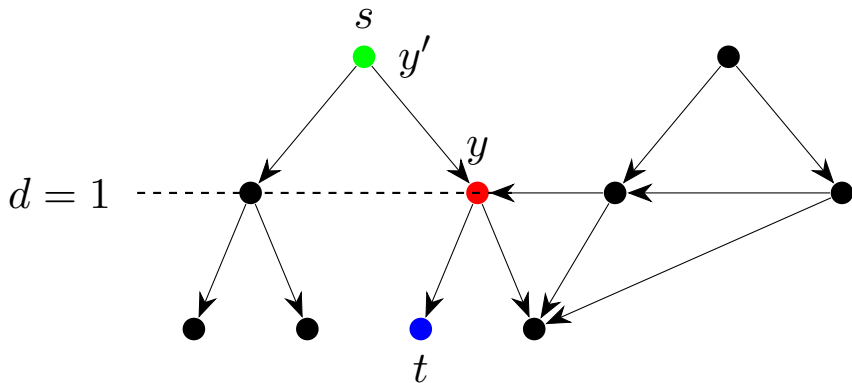


Call **Next**(y', h')

Lange's algorithm informally

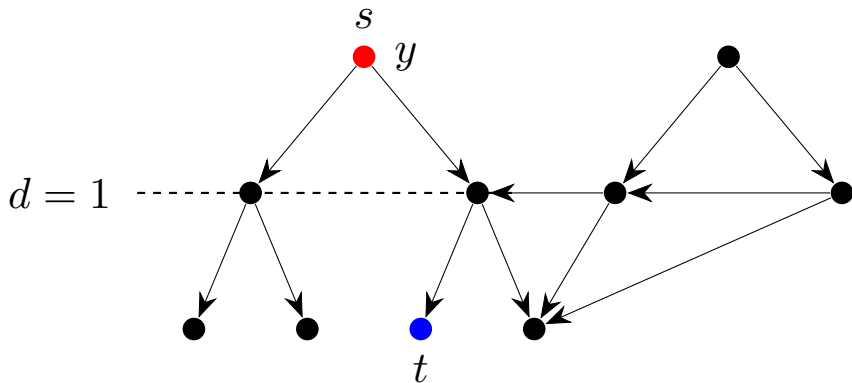


Lange's algorithm informally

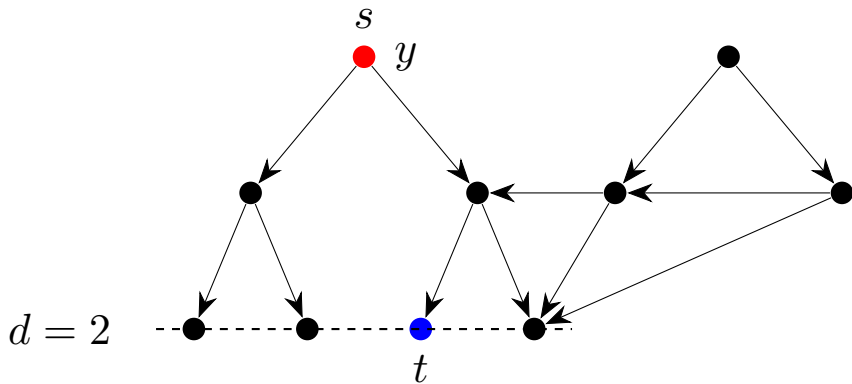


Call Next(y, h)

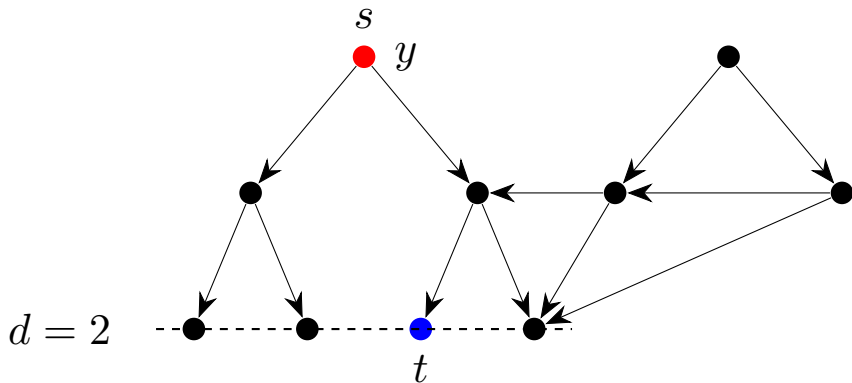
Lange's algorithm informally



Lange's algorithm informally

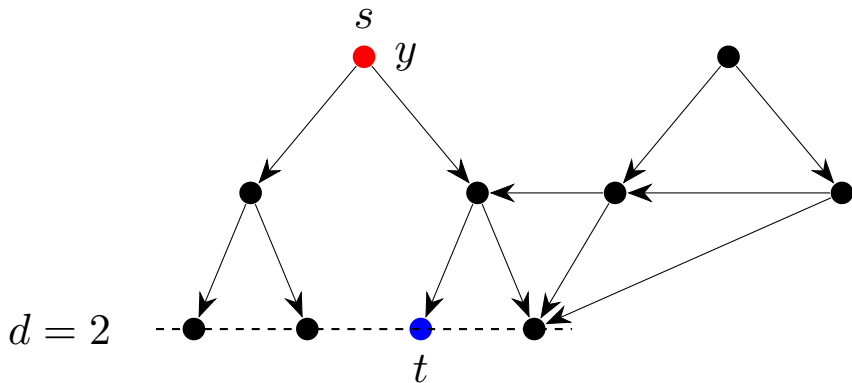


Lange's algorithm informally



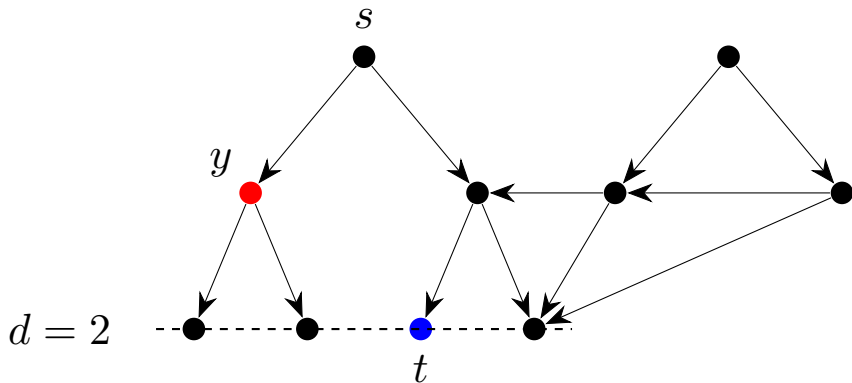
Pre-order traversal for y starts.

Lange's algorithm informally

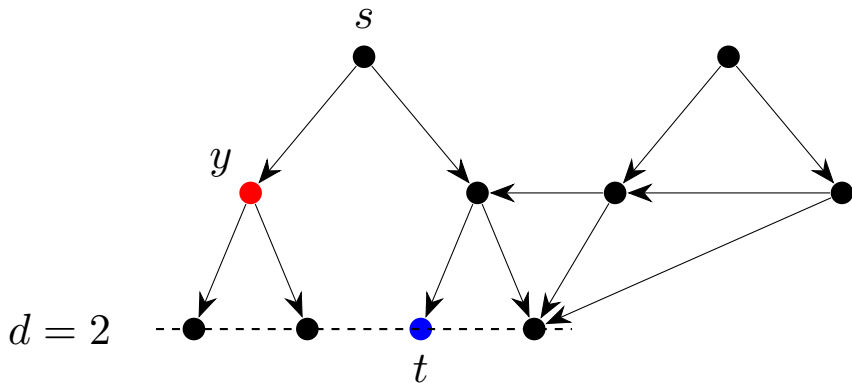


Call **Next**(y, h)

Lange's algorithm informally

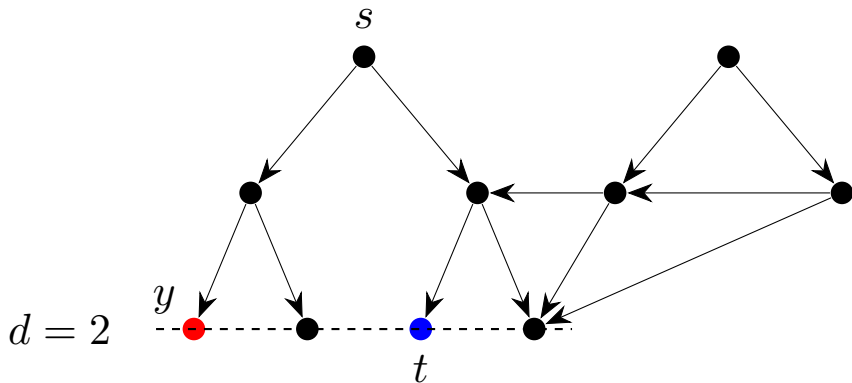


Lange's algorithm informally

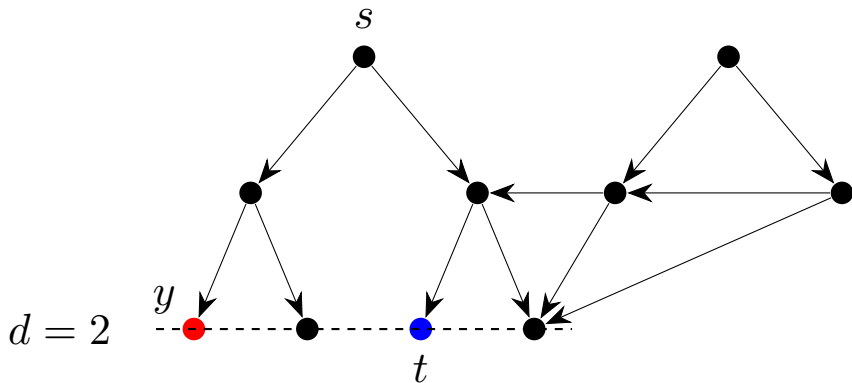


Call $\text{Next}(y, h)$

Lange's algorithm informally

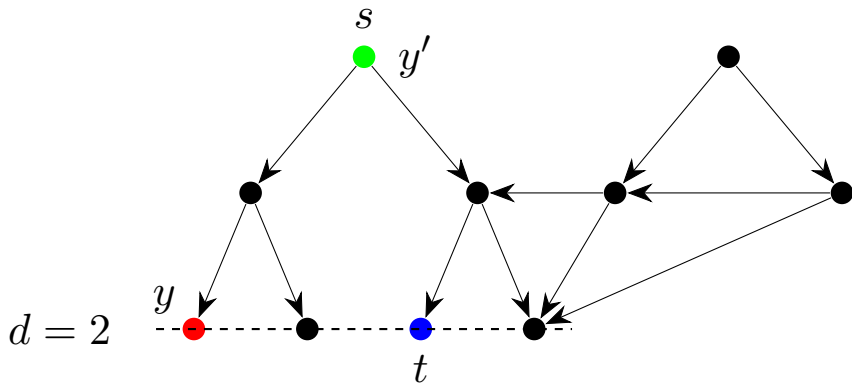


Lange's algorithm informally

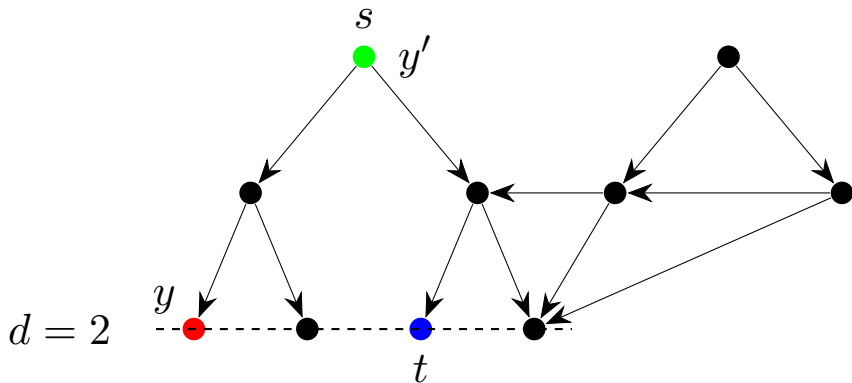


Pre-order traversal for y' starts

Lange's algorithm informally

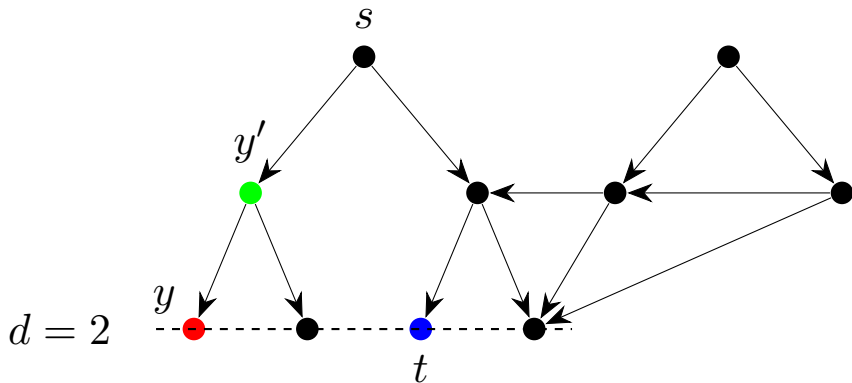


Lange's algorithm informally

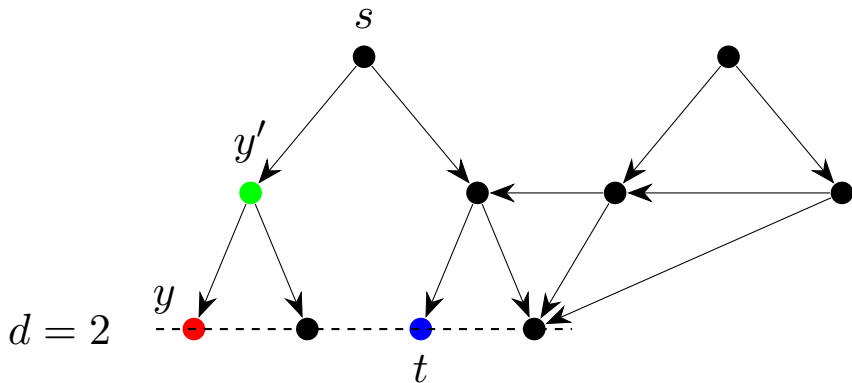


Call **Next**(y', h')

Lange's algorithm informally

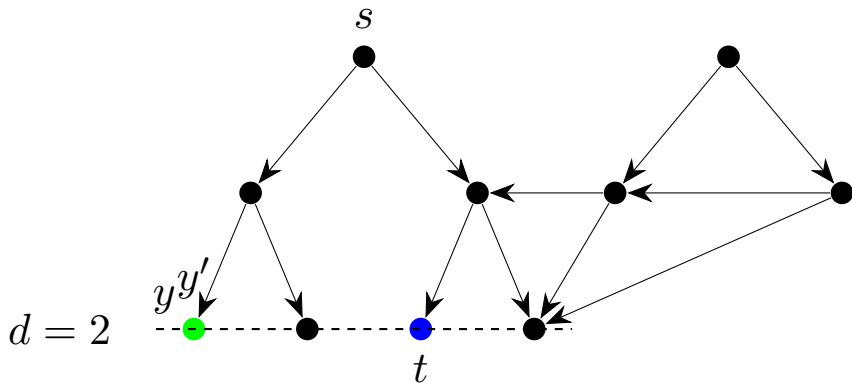


Lange's algorithm informally

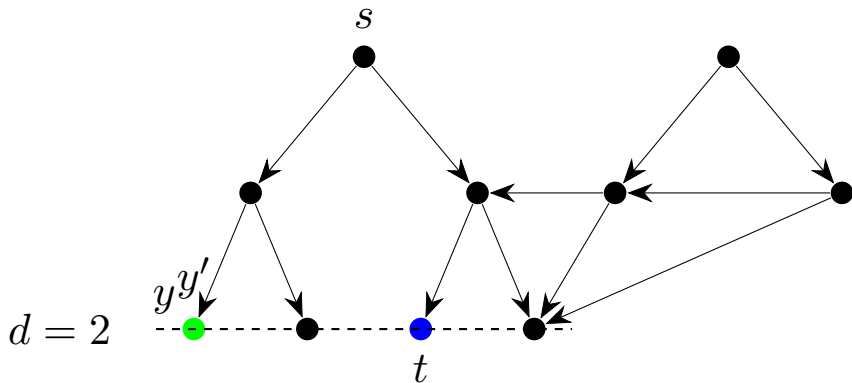


Call **Next**(y', h')

Lange's algorithm informally

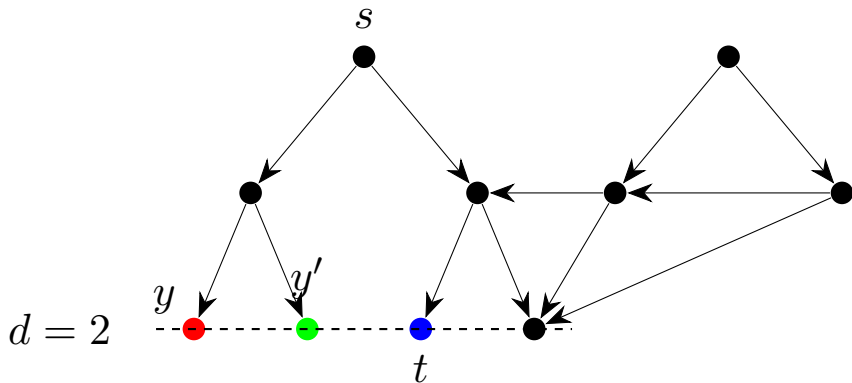


Lange's algorithm informally

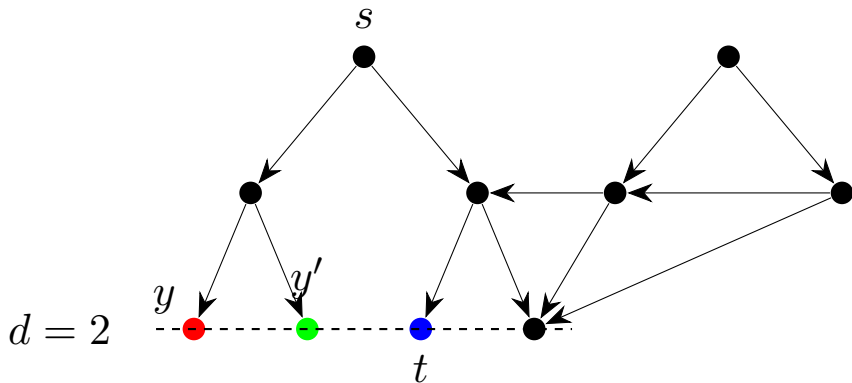


Call **Next**(y', h')

Lange's algorithm informally

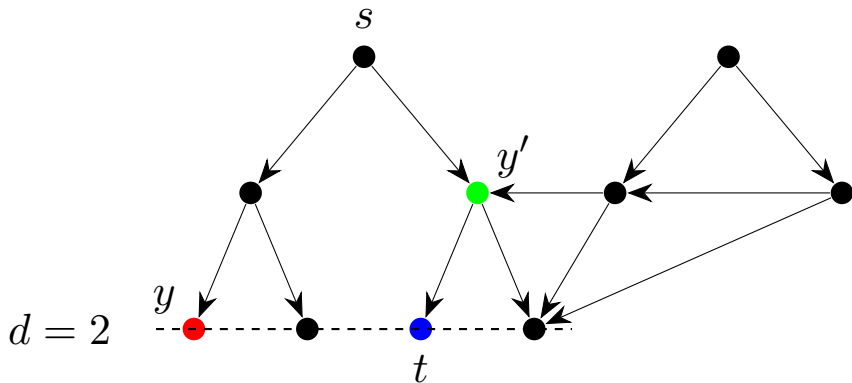


Lange's algorithm informally

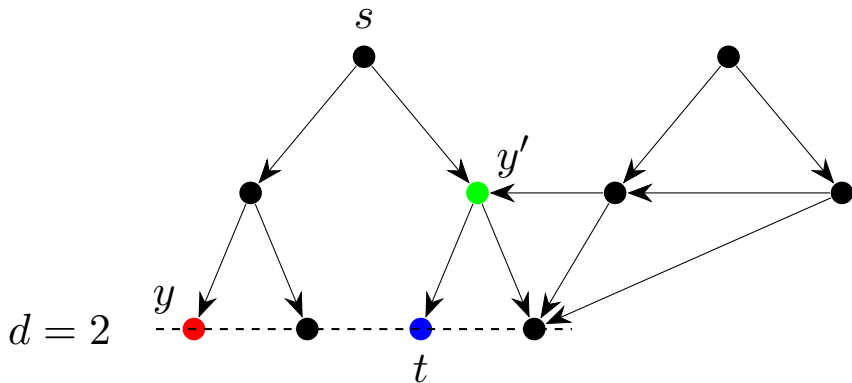


Call **Next**(y' , h')

Lange's algorithm informally

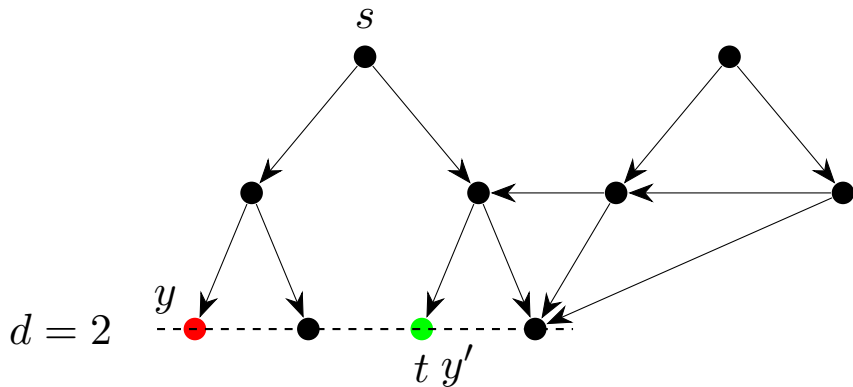


Lange's algorithm informally

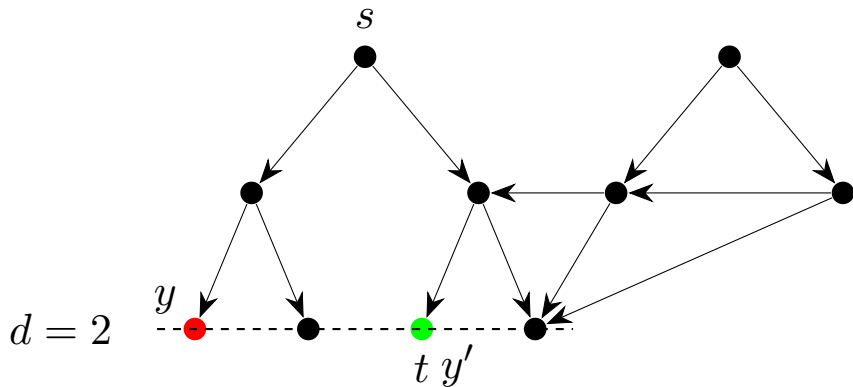


Call **Next**(y', h')

Lange's algorithm informally

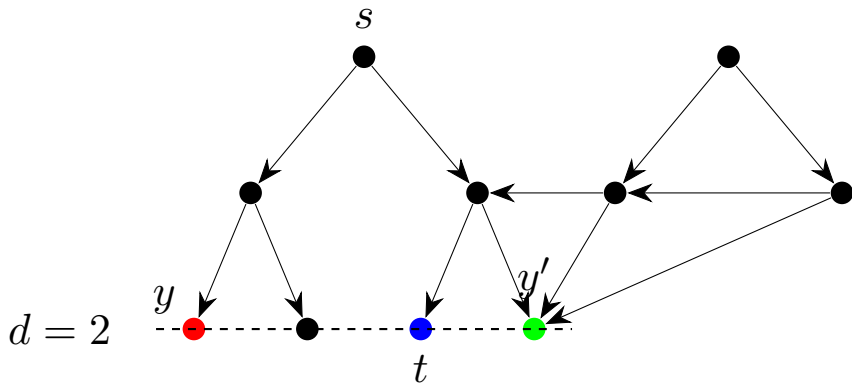


Lange's algorithm informally

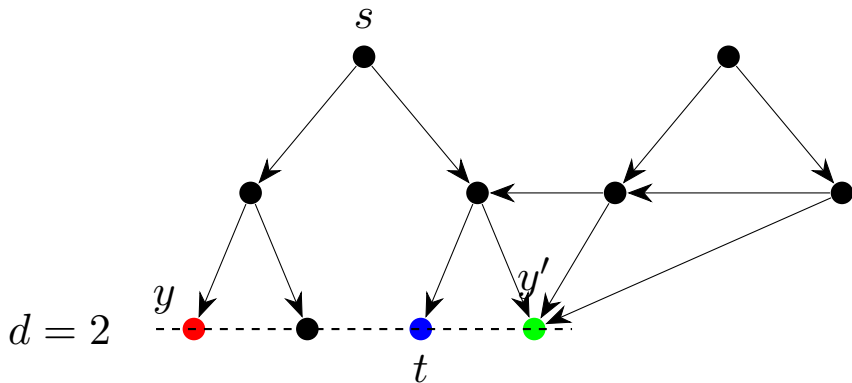


Call **Next**(y', h')

Lange's algorithm informally

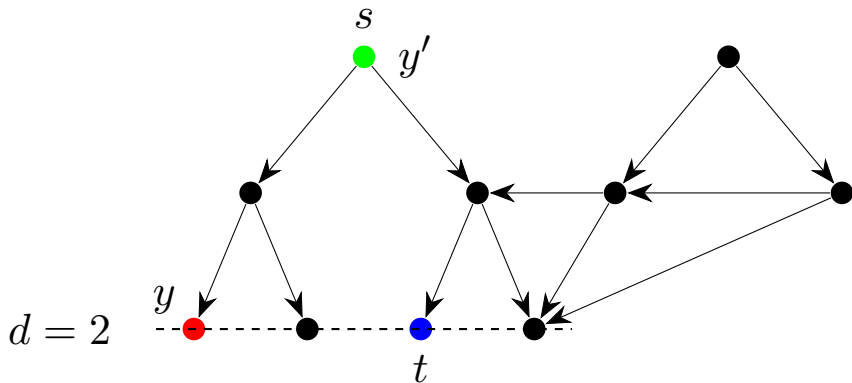


Lange's algorithm informally

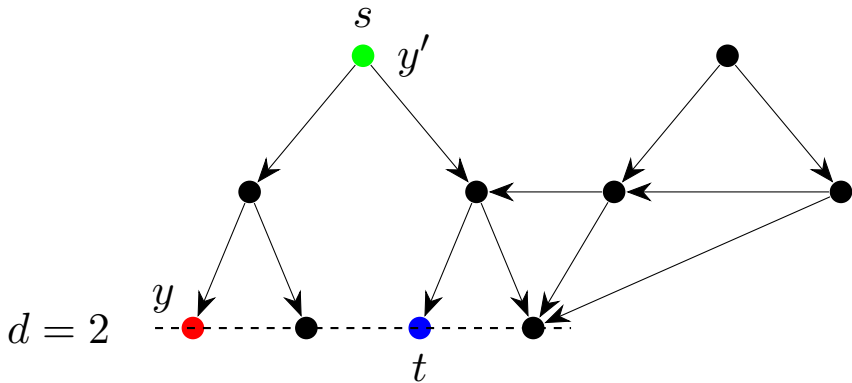


Call **Next**(y' , h')

Lange's algorithm informally

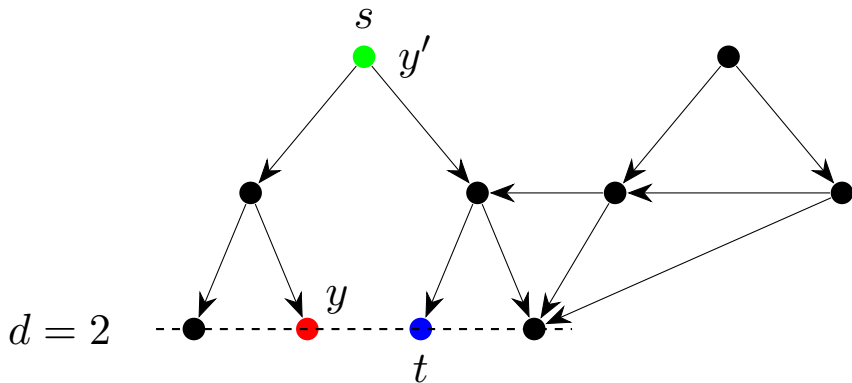


Lange's algorithm informally

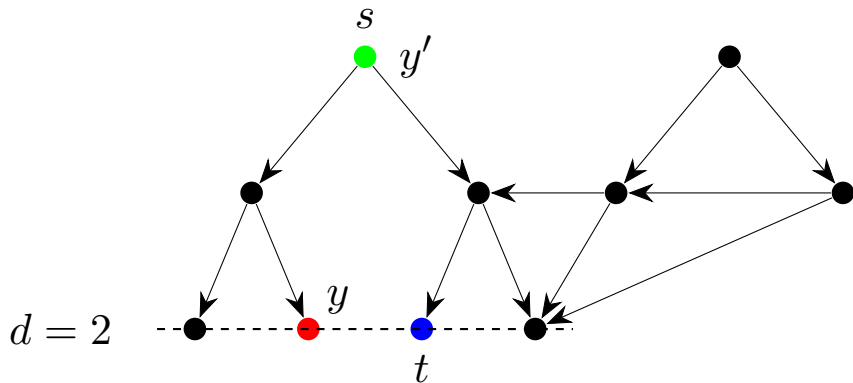


Call Next(y, h)

Lange's algorithm informally

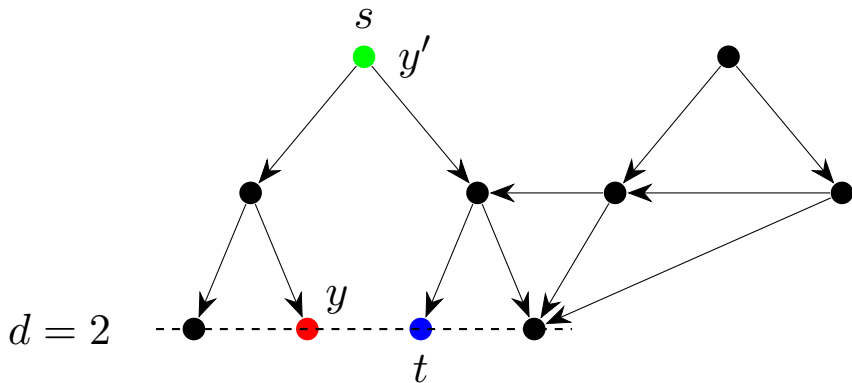


Lange's algorithm informally



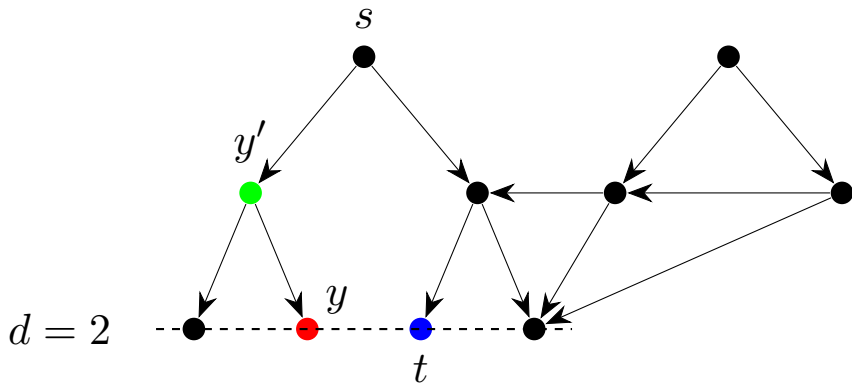
Pre-order traversal for y' starts

Lange's algorithm informally

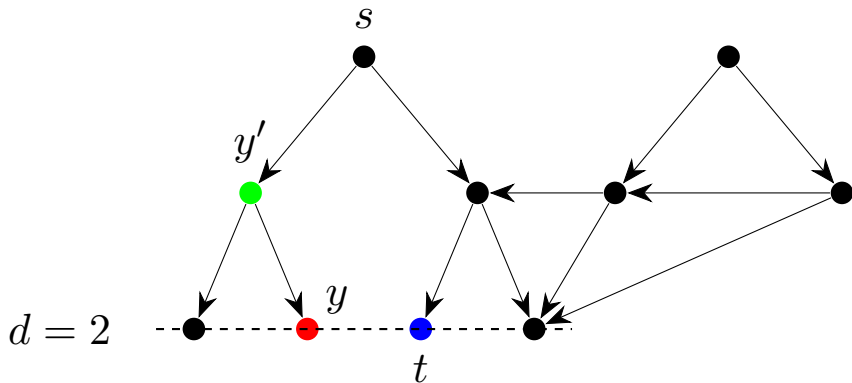


Call **Next**(y', h')

Lange's algorithm informally

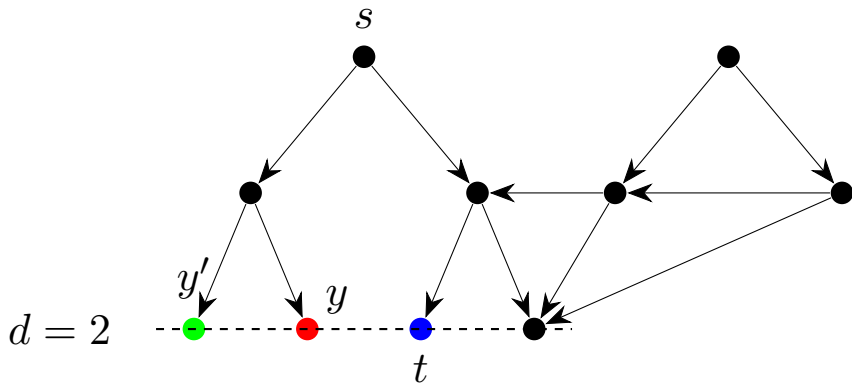


Lange's algorithm informally

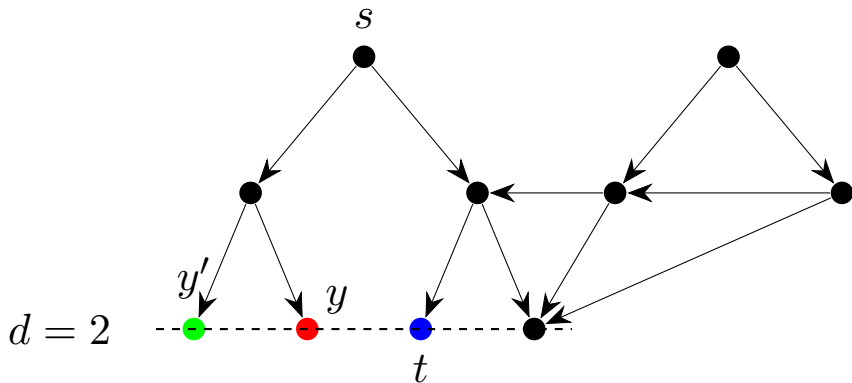


Call **Next**(y', h')

Lange's algorithm informally

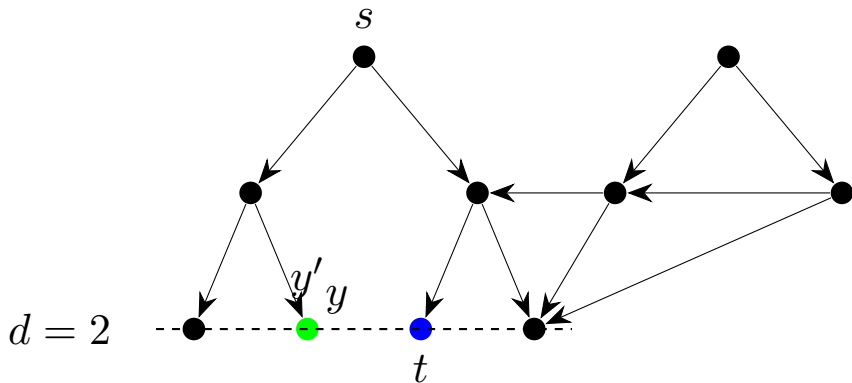


Lange's algorithm informally

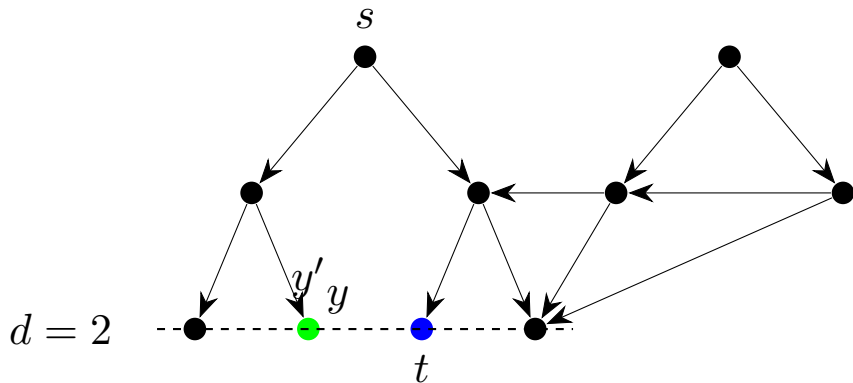


Call **Next**(y', h')

Lange's algorithm informally

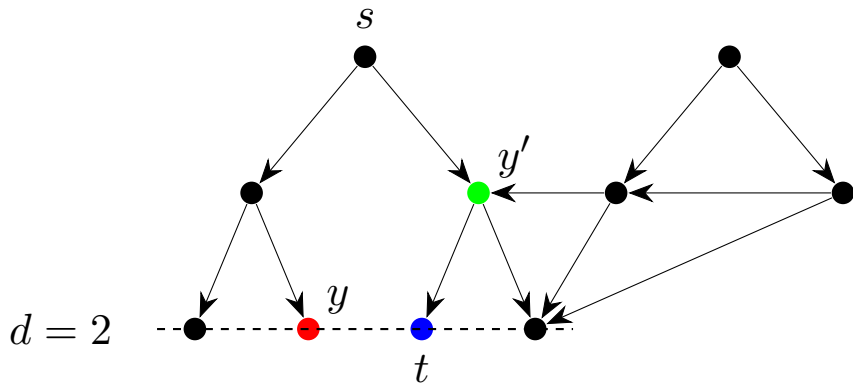


Lange's algorithm informally

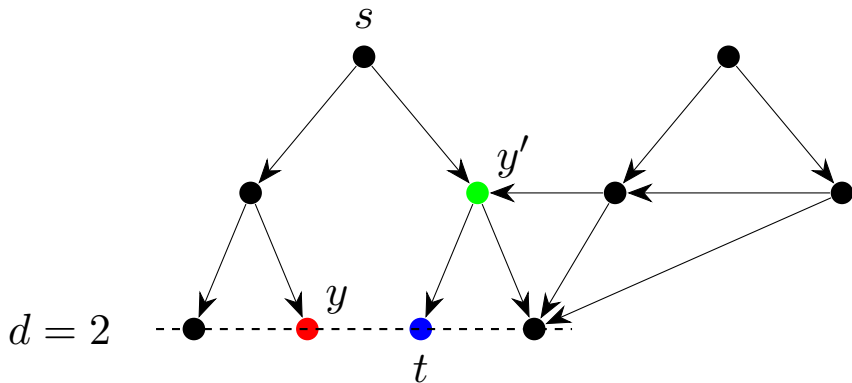


Call **Next**(y' , h')

Lange's algorithm informally

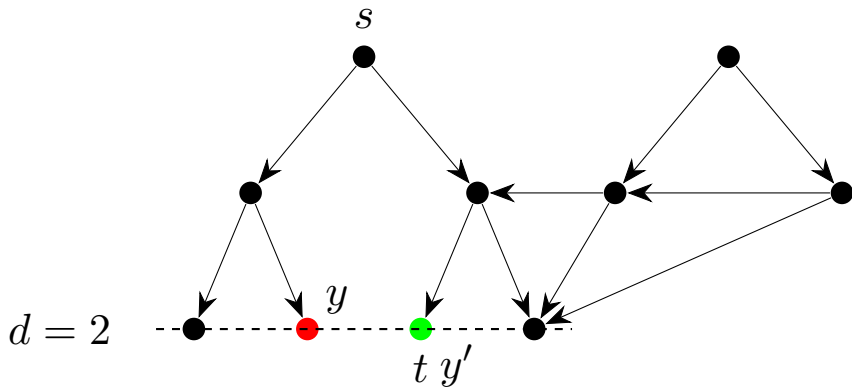


Lange's algorithm informally

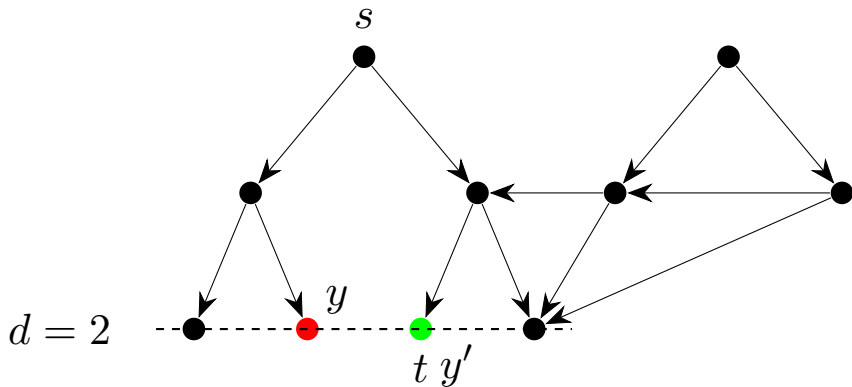


Call **Next**(y', h')

Lange's algorithm informally

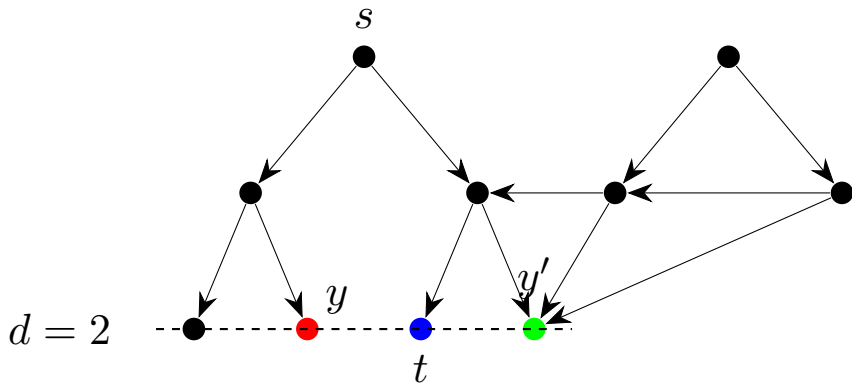


Lange's algorithm informally

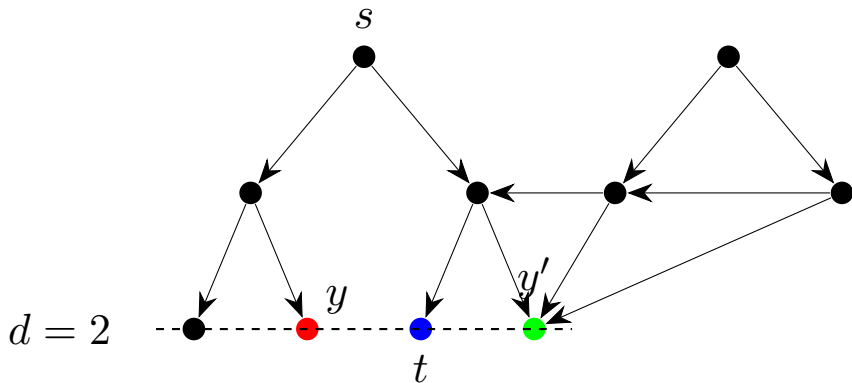


Call **Next**(y', h')

Lange's algorithm informally

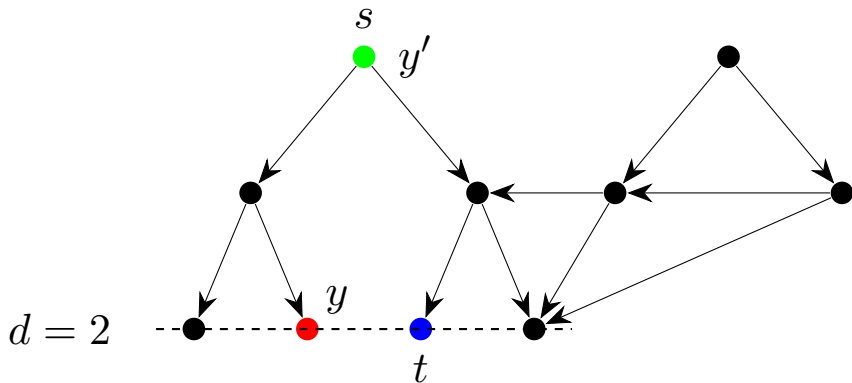


Lange's algorithm informally

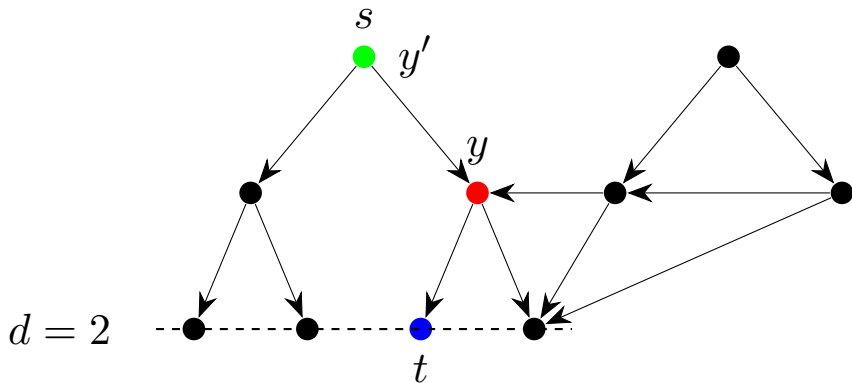


Call **Next**(y', h')

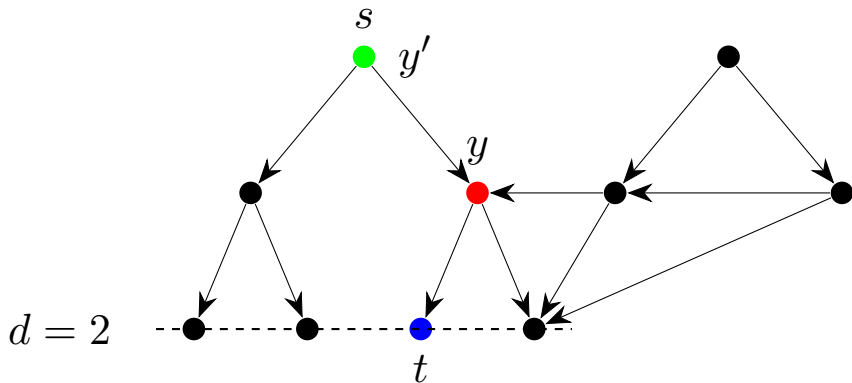
Lange's algorithm informally



Lange's algorithm informally

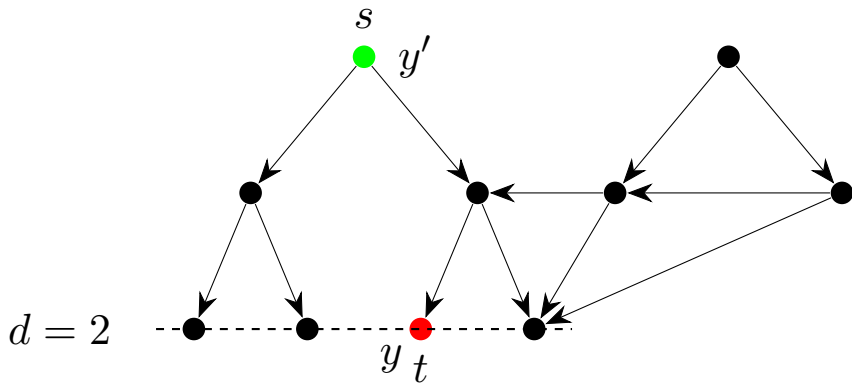


Lange's algorithm informally

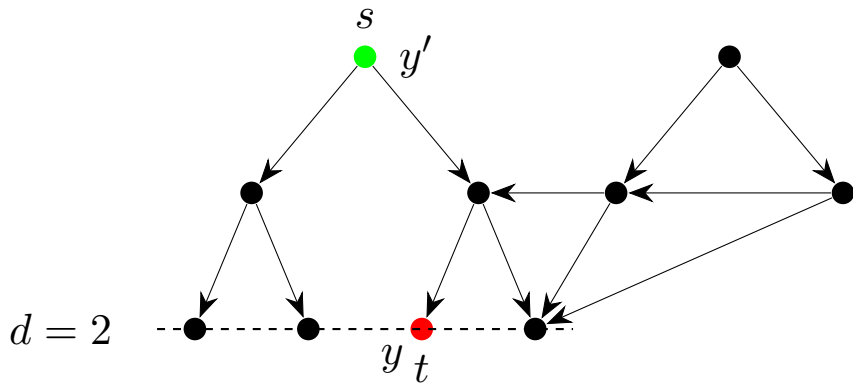


Call Next(y, h)

Lange's algorithm informally

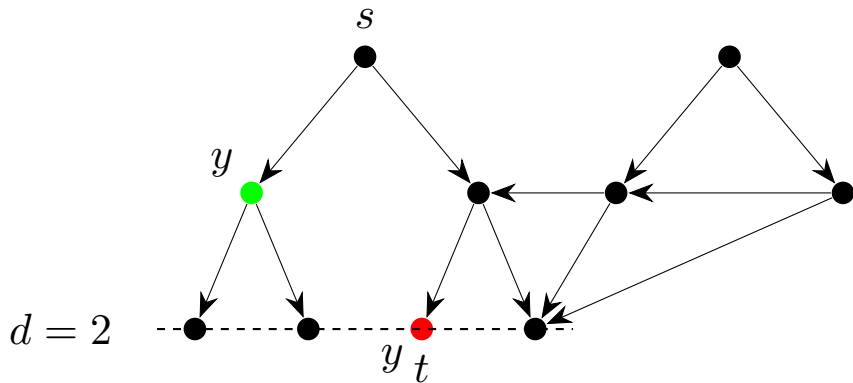


Lange's algorithm informally

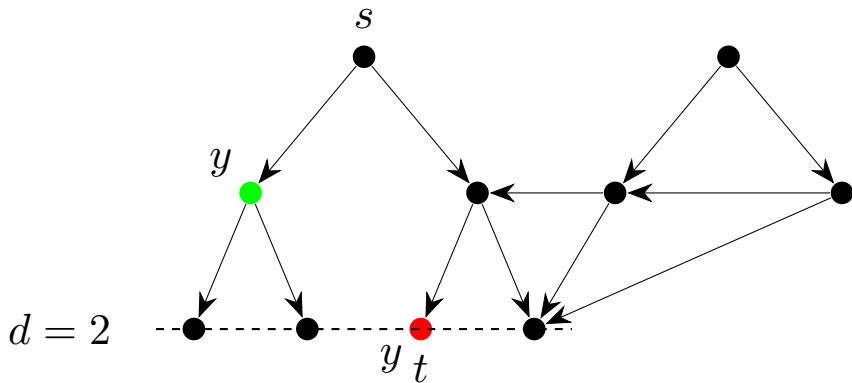


Pre-order traversal for y' starts

Lange's algorithm informally

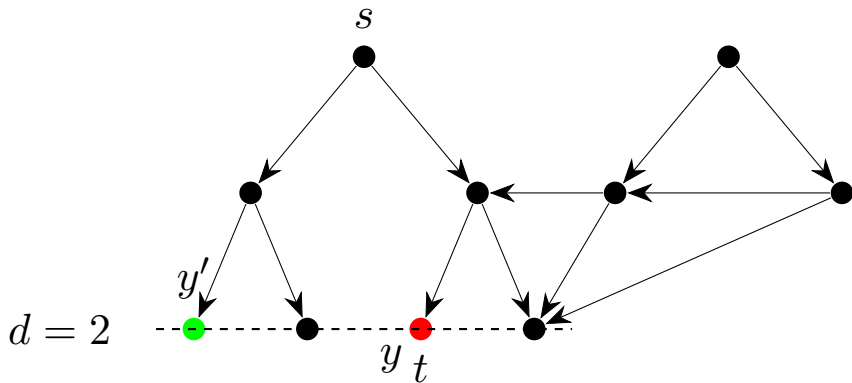


Lange's algorithm informally

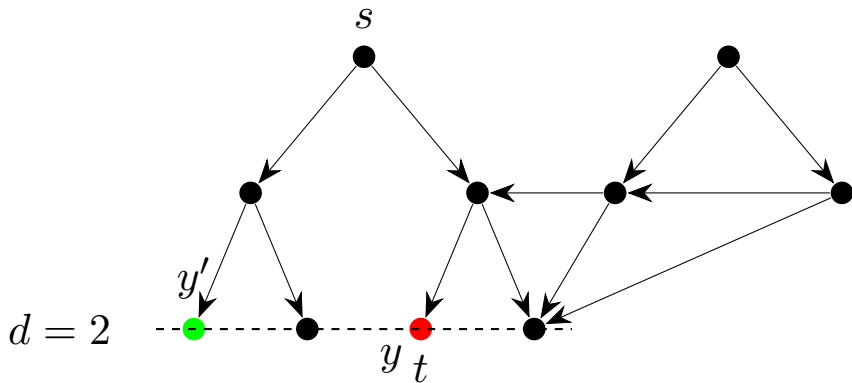


Call Next(y', h')

Lange's algorithm informally

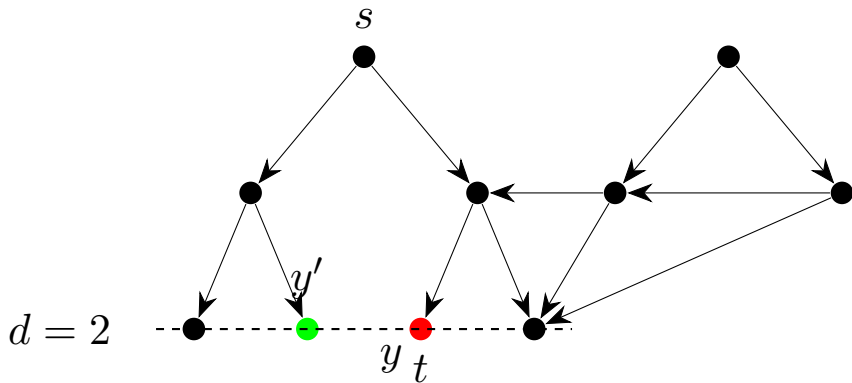


Lange's algorithm informally

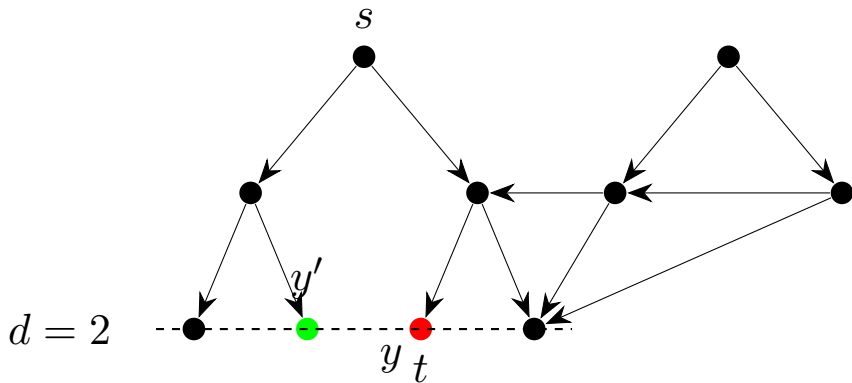


Call **Next**(y', h')

Lange's algorithm informally

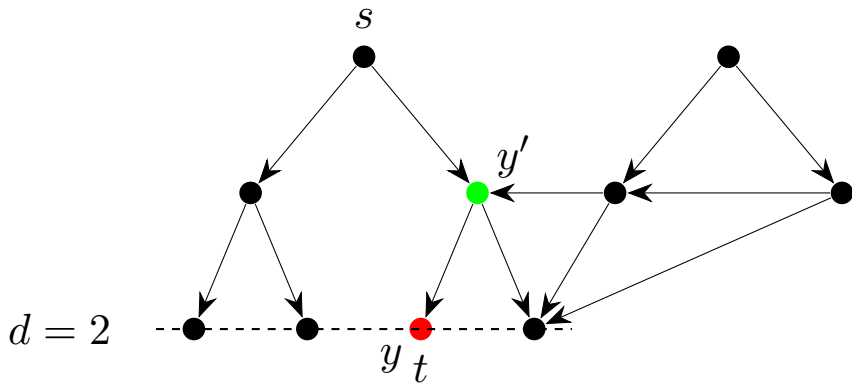


Lange's algorithm informally

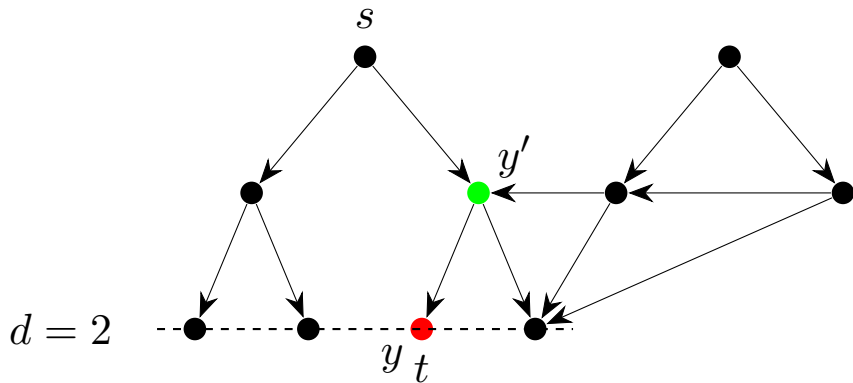


Call Next(y' , h')

Lange's algorithm informally

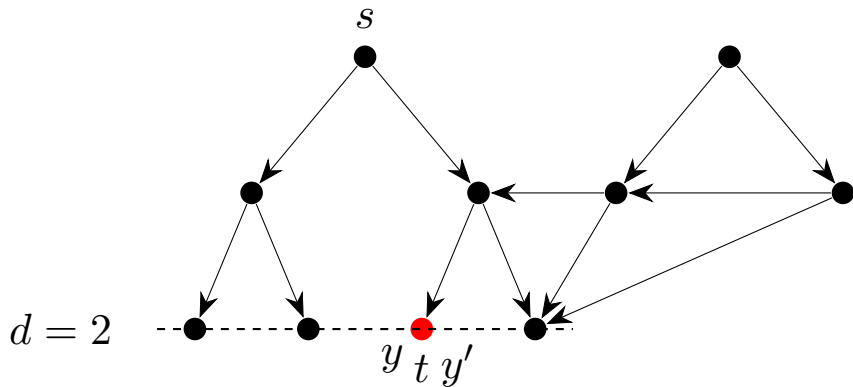


Lange's algorithm informally

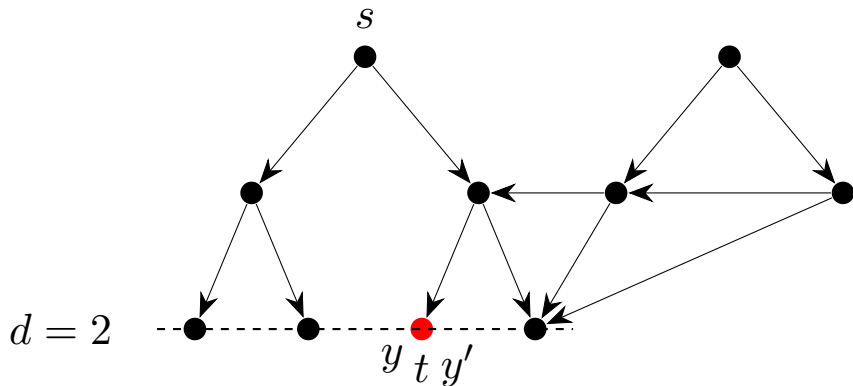


Call **Next**(y', h')

Lange's algorithm informally

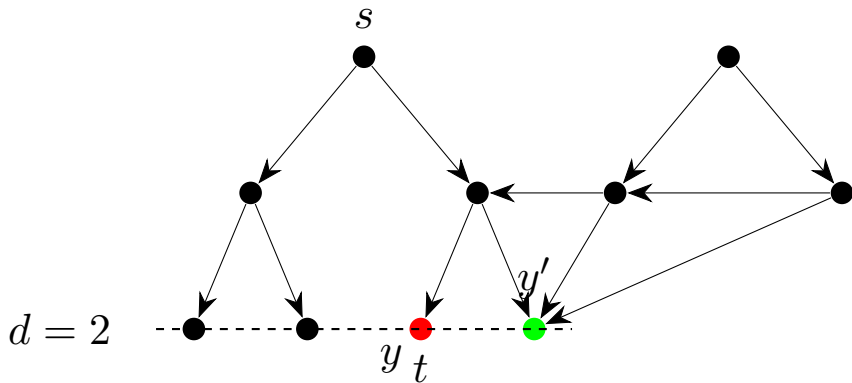


Lange's algorithm informally

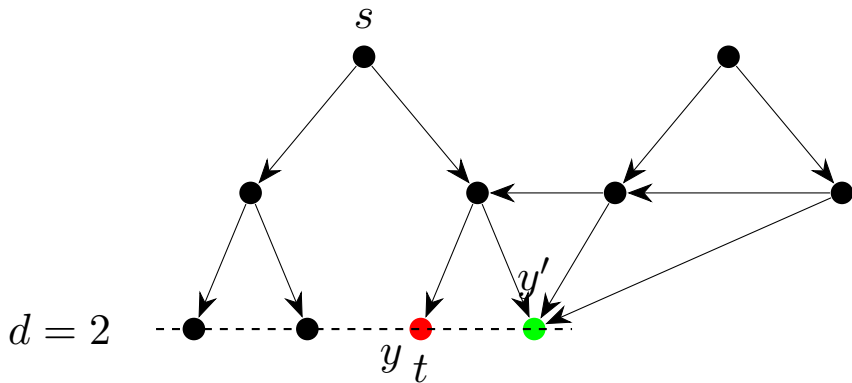


Call **Next**(y', h')

Lange's algorithm informally

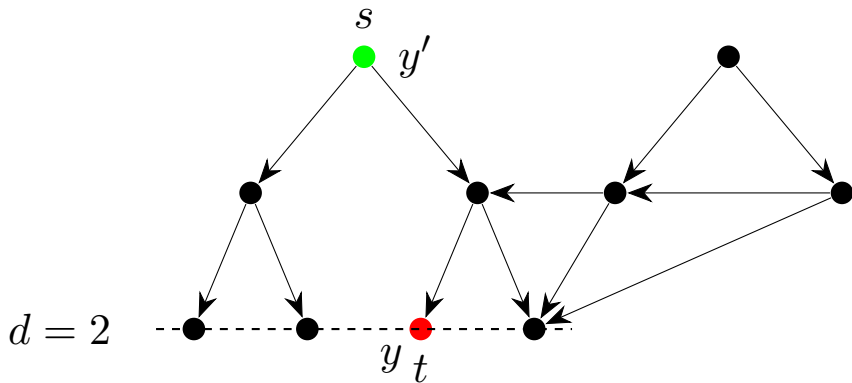


Lange's algorithm informally

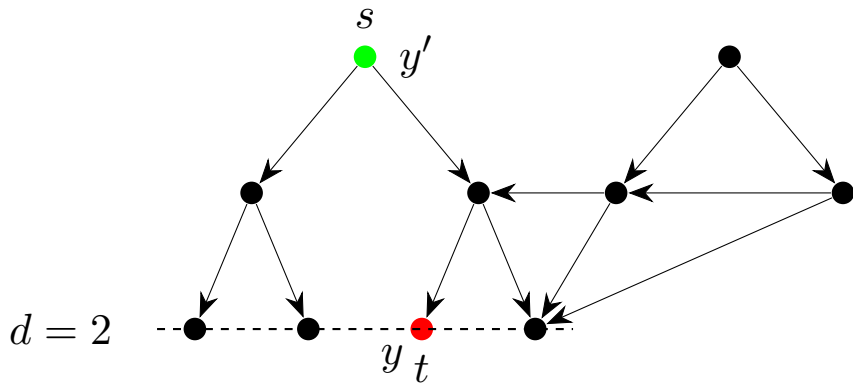


Call **Next**(y', h')

Lange's algorithm informally

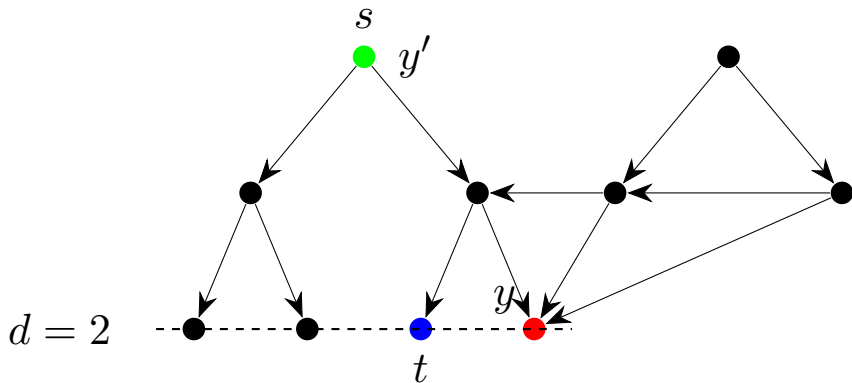


Lange's algorithm informally

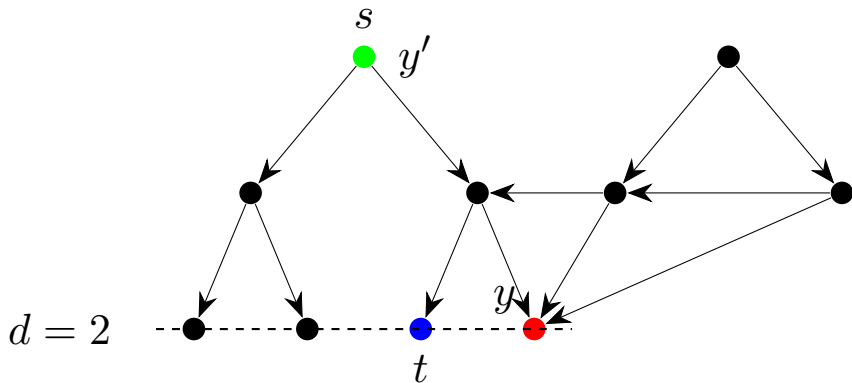


Call **Next**(y', h')

Lange's algorithm informally

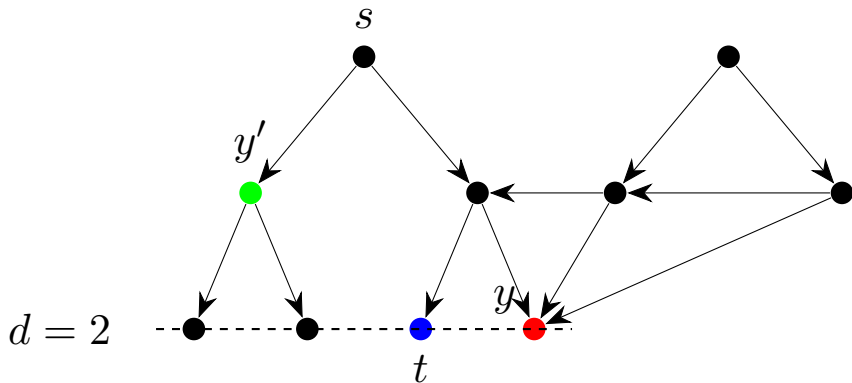


Lange's algorithm informally

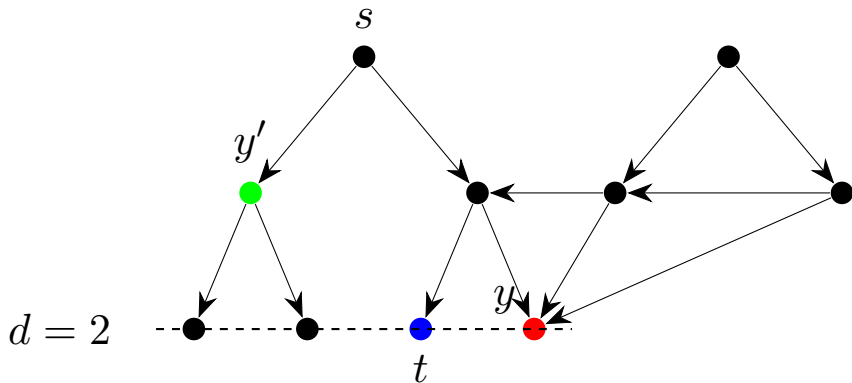


Pre-order traversal for y' starts

Lange's algorithm informally

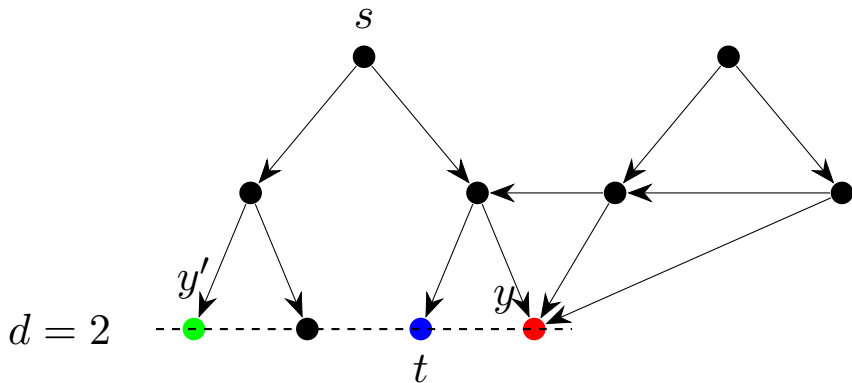


Lange's algorithm informally

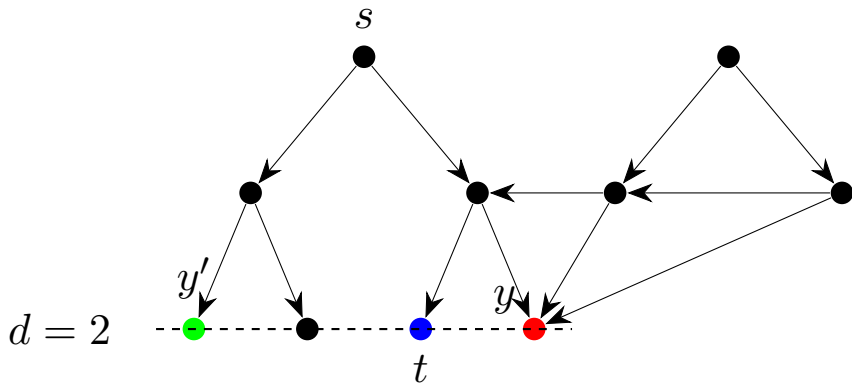


Call **Next**(y' , h')

Lange's algorithm informally

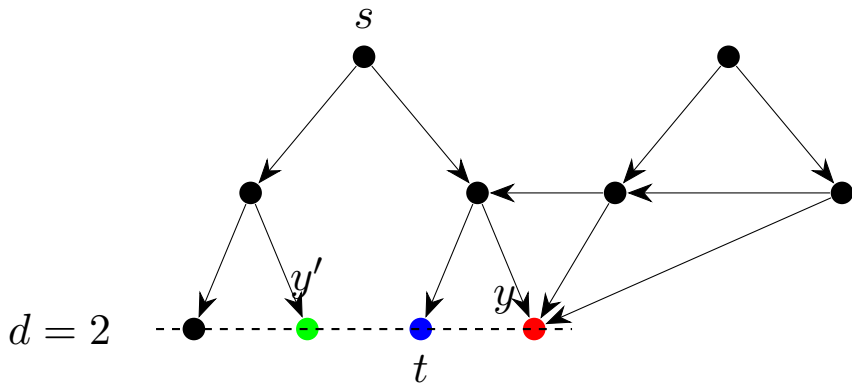


Lange's algorithm informally

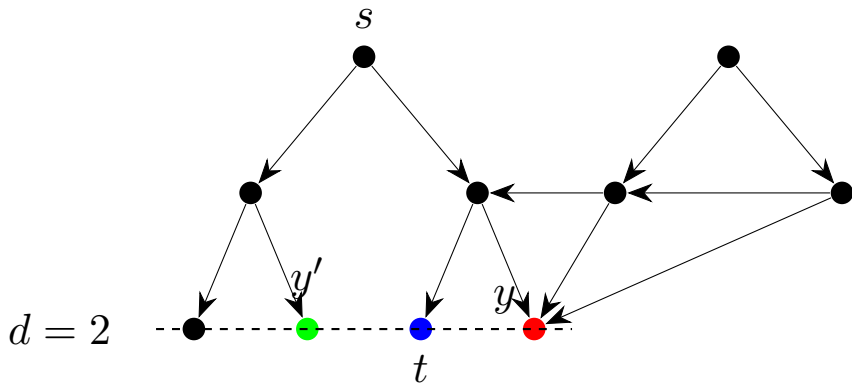


Call **Next**(y', h')

Lange's algorithm informally

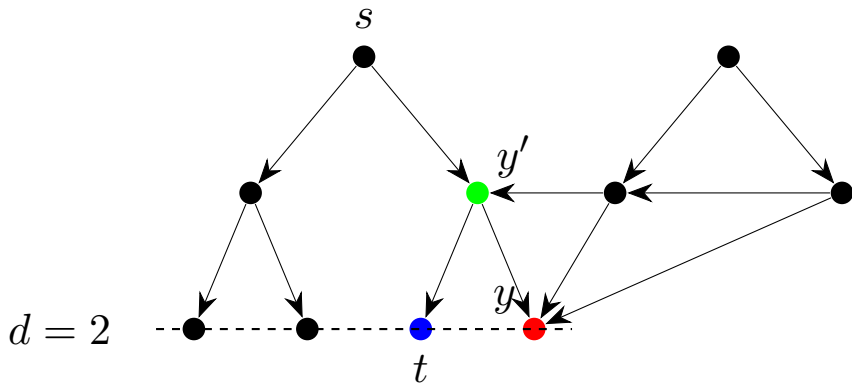


Lange's algorithm informally

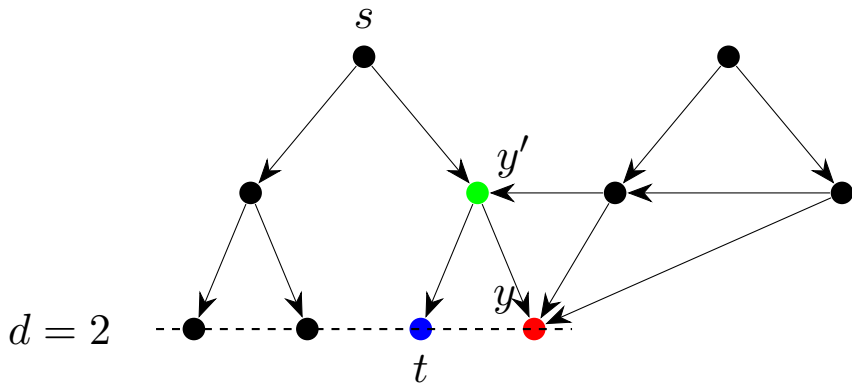


Call **Next**(y' , h')

Lange's algorithm informally

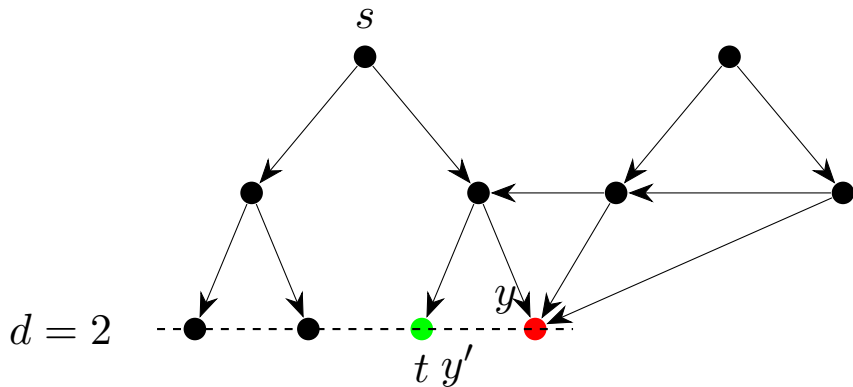


Lange's algorithm informally

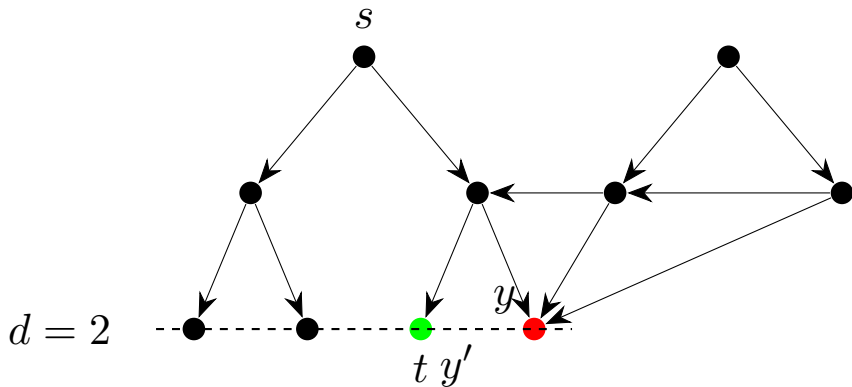


Call **Next**(y', h')

Lange's algorithm informally

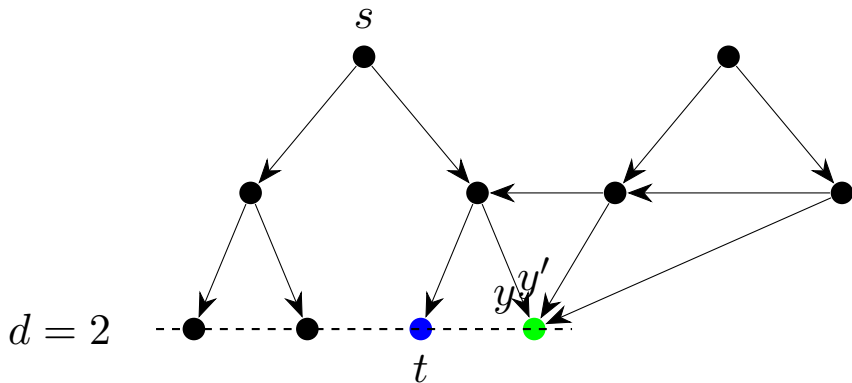


Lange's algorithm informally

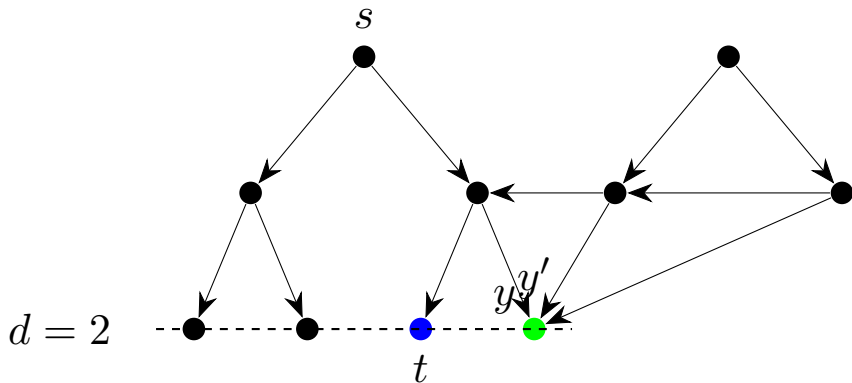


Call **Next**(y', h')

Lange's algorithm informally

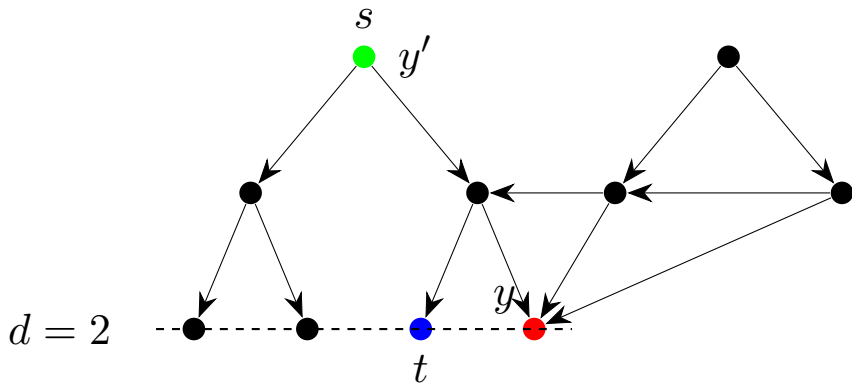


Lange's algorithm informally

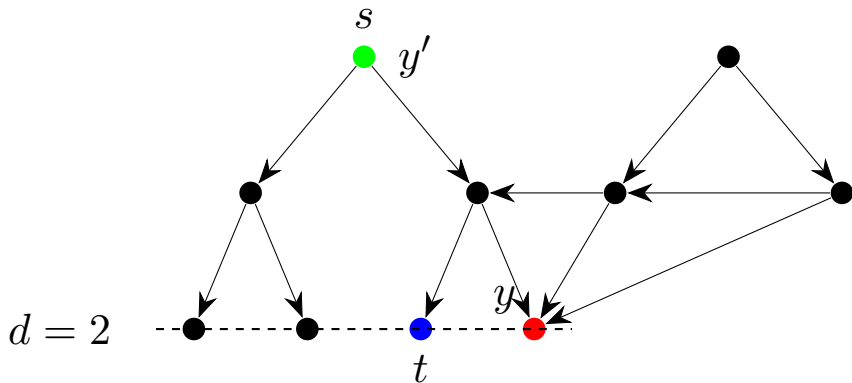


Call **Next**(y' , h')

Lange's algorithm informally

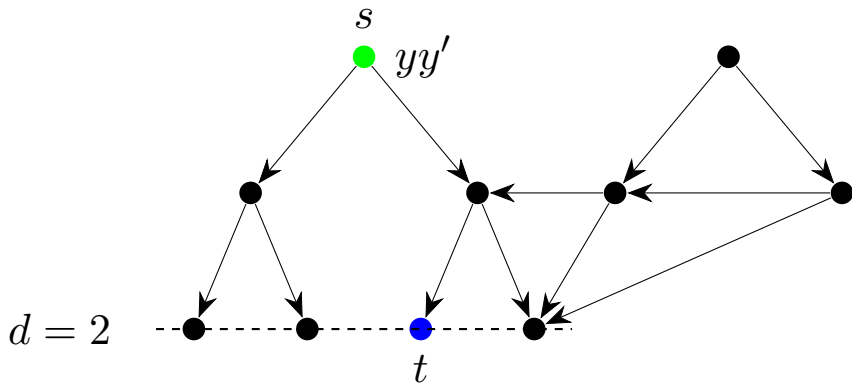


Lange's algorithm informally



Call **Next**(y', h')

Lange's algorithm informally



Lange's Algorithm Informally

For $d = 1$ **to** $n - 2$:

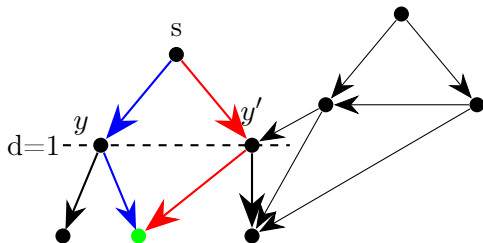
For (y, h) in pre-order traversal of G :

- If $\text{child}(y) = t$, $\text{reached} \leftarrow \text{True}$.
- If y at distance d and is internal node.
 - For each (y', h') in pre-order traversal of $G_d(s)$:
 - If y' is a child of y , **Reject**.
 - If y' at distance d , $y \neq y'$, y' is internal, and share a common child, **Reject**.

Reject Case 1: When y and y' are at distance d .

For each (y', h') in pre-order traversal of $G_d(s)$:

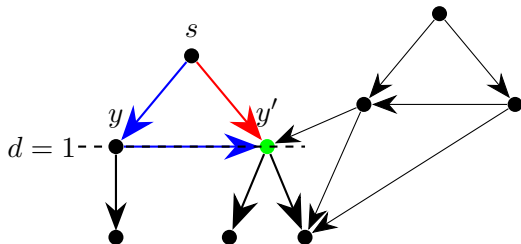
- If y' is a child of y ,
Reject.
- If y' at distance d ,
 $y \neq y'$, y' is internal,
and share a common
child, **Reject.**



Reject Case 2: When y' is a child of y .

For each (y', h') in pre-order traversal of $G_d(s)$:

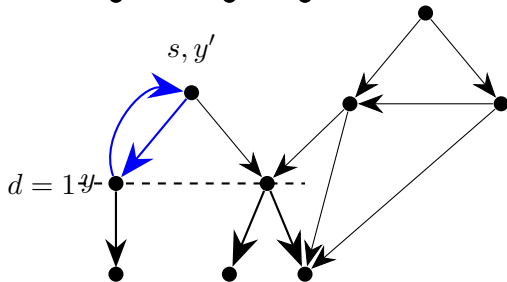
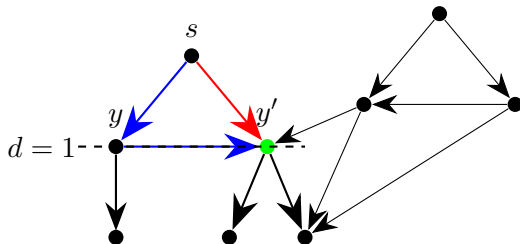
- If y' is a child of y , **Reject**.
- If y' at distance d , $y \neq y'$, y' is internal, and share a common child, **Reject**.



Reject Case 2: When y' is a child of y .

For each (y', h') in pre-order traversal of $G_d(s)$:

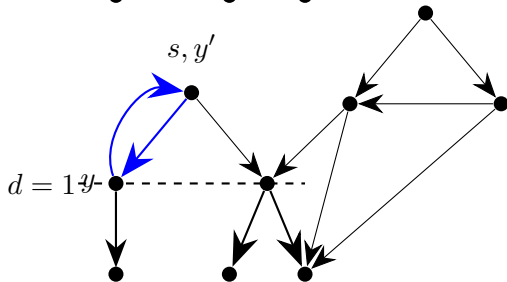
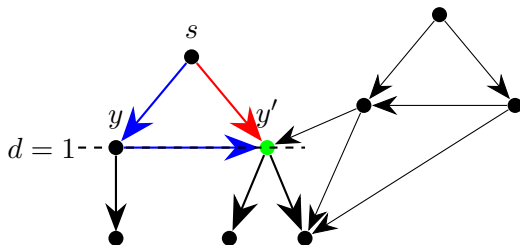
- If y' is a child of y , **Reject**.
- If y' at distance d , $y \neq y'$, y' is internal, and share a common child, **Reject**.



Reject Case 2: When y' is a child of y .

For each (y', h') in pre-order traversal of $G_d(s)$:

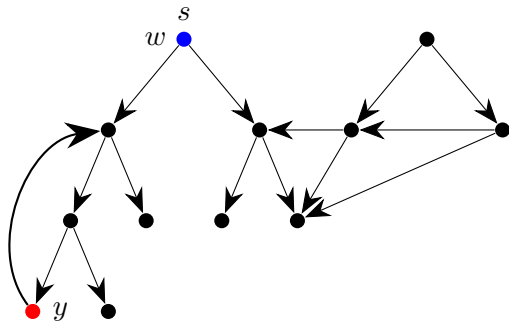
- If y' is a child of y , **Reject**.
- If y' at distance d , $y \neq y'$, y' is internal, and share a common child, **Reject**.



Challenge 1: Do not want to reject on back edges

How do we detect back edges?

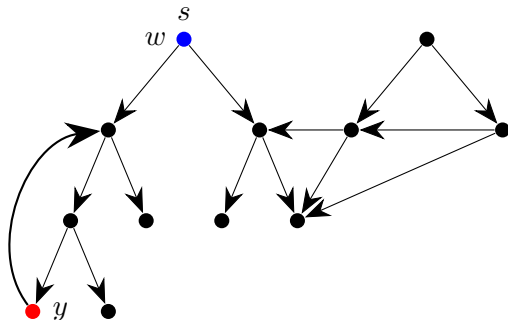
Detecting Left Back Edges



Flag=False

How do we detect back edges?

Detecting Left Back Edges

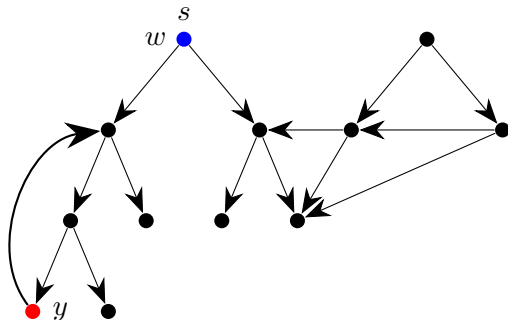


Flag=False

If w left child of y , set Flag=True.

How do we detect back edges?

Detecting Left Back Edges

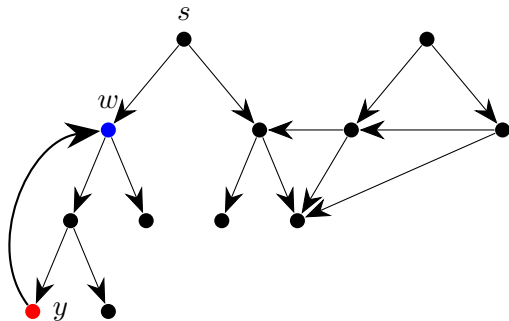


Guess next neighbour of w

Flag=False

How do we detect back edges?

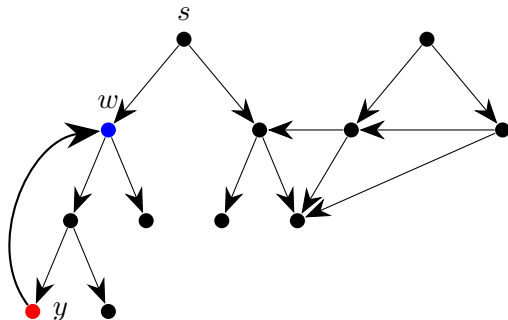
Detecting Left Back Edges



Flag=False

How do we detect back edges?

Detecting Left Back Edges

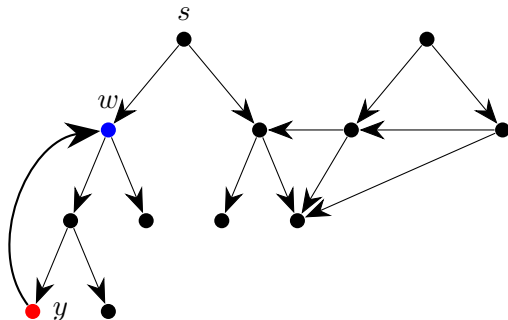


Flag=True

If w left child of y , set Flag=True.

How do we detect back edges?

Detecting Left Back Edges

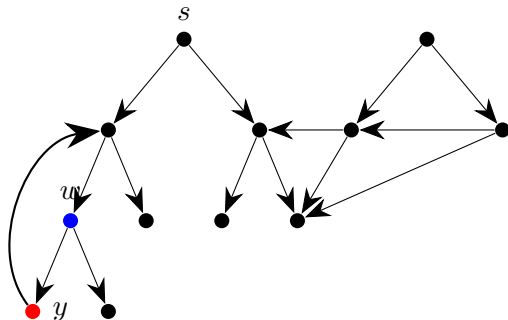


Guess next neighbour of w

Flag=True

How do we detect back edges?

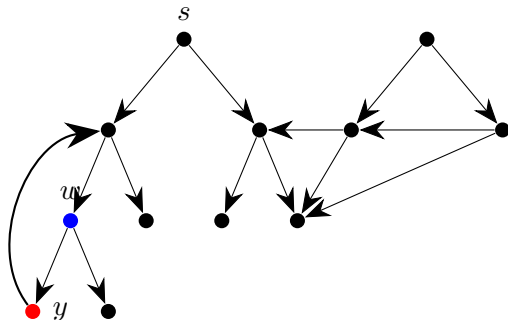
Detecting Left Back Edges



Flag=True

How do we detect back edges?

Detecting Left Back Edges

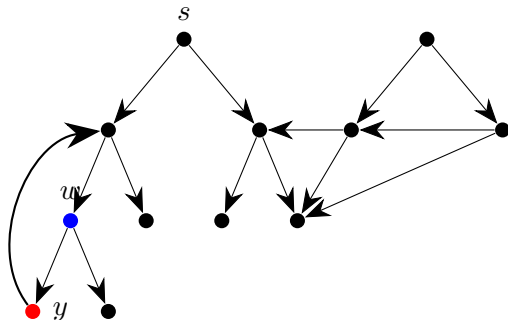


If w left child of y , set **Flag=True**.

Flag=True

How do we detect back edges?

Detecting Left Back Edges

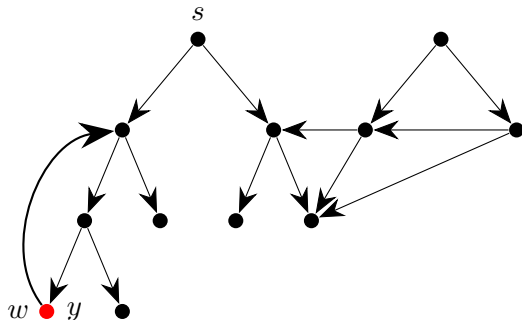


Guess next neighbour of w

Flag=True

How do we detect back edges?

Detecting Left Back Edges



Flag=True

For $d = 1$ **to** $n - 2$:

For (y, h) in pre-order traversal of G :

- If $\text{child}(y) = t$, $\text{reached} \leftarrow \text{True}$.
- If y at distance d and is internal node.
 - For each (y', h') in pre-order traversal of $G_d(s)$:
 - If y' is a child of y and **not a back edge**, **Reject**.
 - If y' at distance d , $y \neq y'$, y' is internal, and share a common child, **Reject**.

- Lange's algorithm: reachability in reach unambiguous graphs.

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.

Does the algorithm work for reachability in simple path graphs now?

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.

Does the algorithm work for reachability in simple path graphs now?

Not yet!

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.

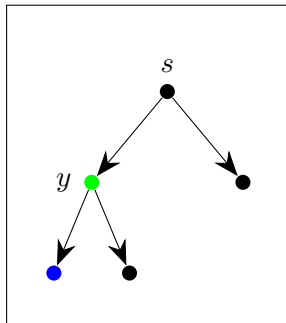
Does the algorithm work for reachability in simple path graphs now?

Not yet!

Next task:

How to correctly compute pre-order successor.

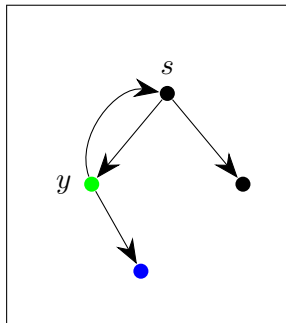
How does pre-order successor appear?



Pre-order successor of y :

- $L(y)$ if $(y, L(y))$ is not a back edge.

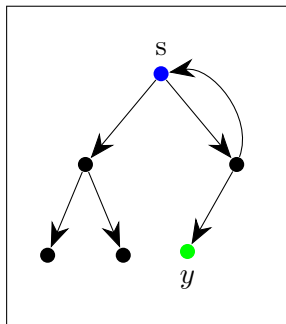
How does pre-order successor appear?



Pre-order successor of y :

- $L(y)$ if $(y, L(y))$ is not a back edge.
- If $L(y)$ is a back edge, $R(y)$ provided $(y, R(y))$ is not a back edge.

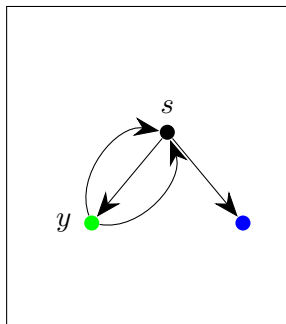
How does pre-order successor appear?



Pre-order successor of y :

- $L(y)$ if $(y, L(y))$ is not a back edge.
- If $L(y)$ is a back edge, $R(y)$ provided $(y, R(y))$ is not a back edge.
- **Challenge 2:** If y is the ‘rightmost’ vertex, s must be returned.

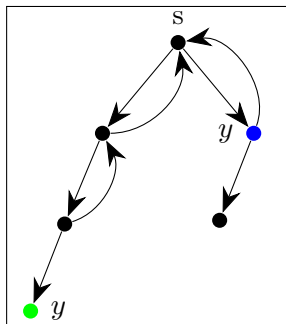
How does pre-order successor appear?



Pre-order successor of y :

- $L(y)$ if $(y, L(y))$ is not a back edge.
- If $L(y)$ is a back edge, $R(y)$ provided $(y, R(y))$ is not a back edge.
- **Challenge 2:** If y is the ‘rightmost’ vertex, s must be returned.
- **Challenge 3:** If $L(y)$ and $R(y)$ are back edges or y is a leaf, pre-order successor needs to be computed.

How does pre-order successor appear?

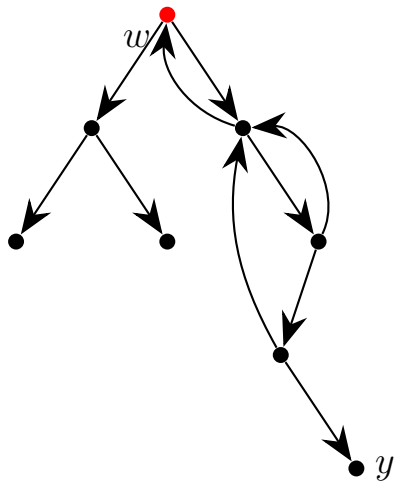


Pre-order successor of y :

- $L(y)$ if $(y, L(y))$ is not a back edge.
- If $L(y)$ is a back edge, $R(y)$ provided $(y, R(y))$ is not a back edge.
- **Challenge 2:** If y is the ‘rightmost’ vertex, s must be returned.
- **Challenge 3:** If $L(y)$ and $R(y)$ are back edges or y is a leaf, pre-order successor needs to be computed.

Addressing Challenge 2: Check if y is rightmost?

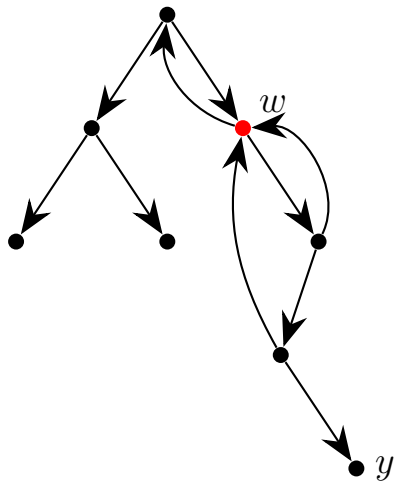
(working of Next)



Go right

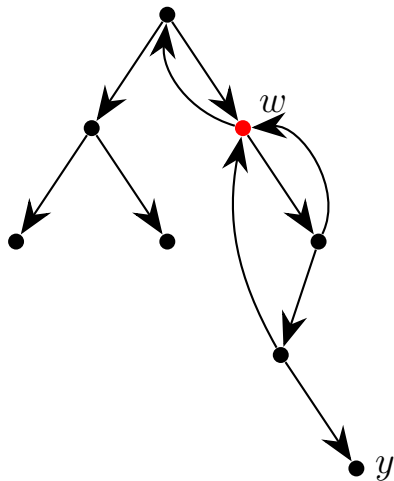
Addressing Challenge 2: Check if y is rightmost?

(working of Next)



Addressing Challenge 2: Check if y is rightmost?

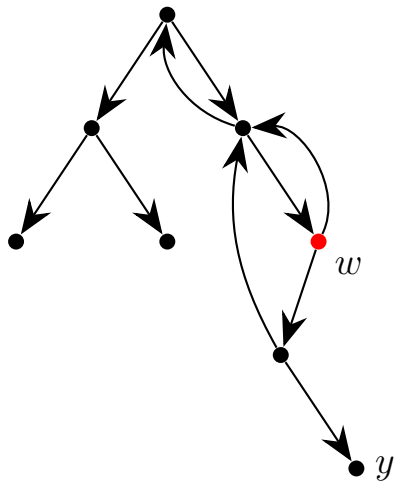
(working of Next)



Go right

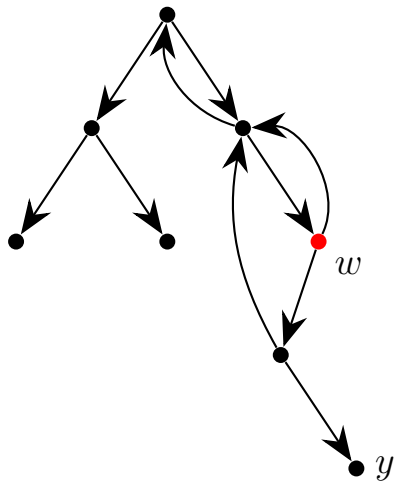
Addressing Challenge 2: Check if y is rightmost?

(working of Next)



Addressing Challenge 2: Check if y is rightmost?

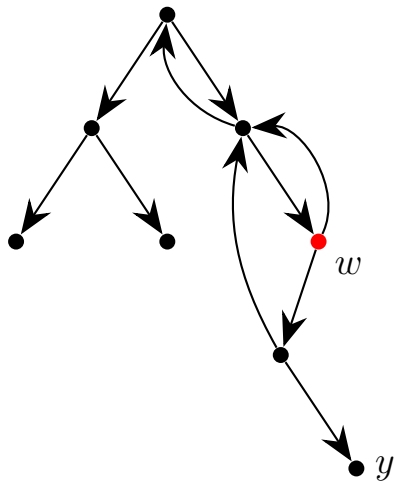
(working of Next)



Go right

Addressing Challenge 2: Check if y is rightmost?

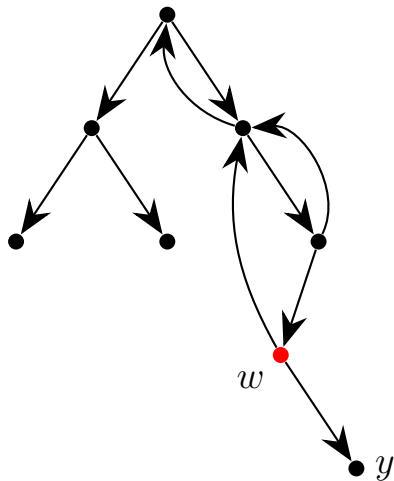
(working of Next)



Can't go right
anymore! Go
left.

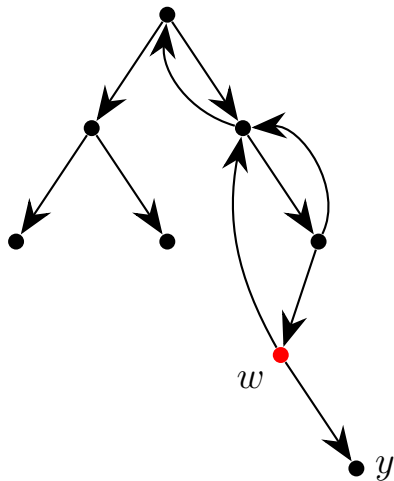
Addressing Challenge 2: Check if y is rightmost?

(working of Next)



Addressing Challenge 2: Check if y is rightmost?

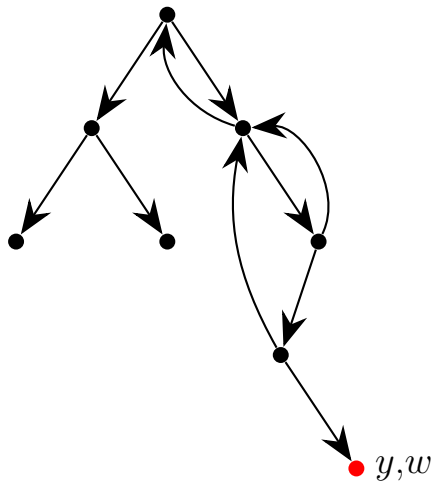
(working of Next)



Go right

Addressing Challenge 2: Check if y is rightmost?

(working of Next)



y is the
rightmost.

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.

Does the algorithm now work for reachability in simple path graphs ?

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.

Does the algorithm now work for reachability in simple path graphs ?

Not yet!

Story so far

- Lange's algorithm: reachability in reach unambiguous graphs.

Challenges

- **Challenge 1:** Rejects on back edge.
 - Modification: Detecting back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.

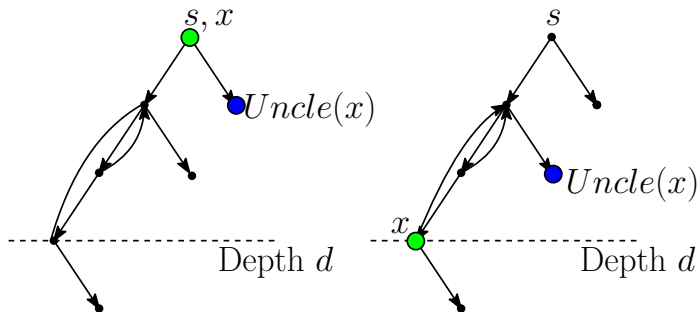
Does the algorithm now work for reachability in simple path graphs ?

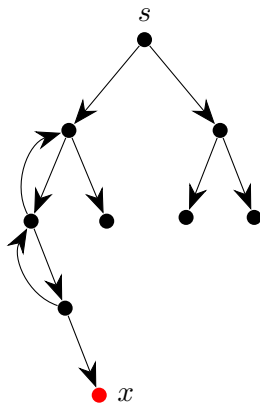
Not yet!

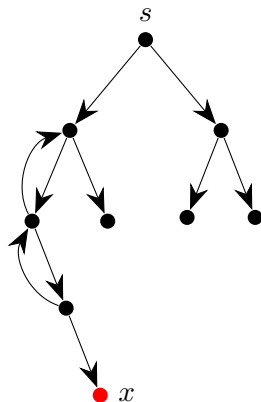
Challenge 3: How to correctly compute pre-order successor when current vertex is a leaf or both its children are back edges.

Who is an Uncle?

$$\text{Uncle}(x) = \begin{cases} R(x) & \text{if } x = s \\ \text{Next vertex to be visited in the pre-order traversal of subtree } G_d(s), \text{ assuming all the vertices of the subtree of } G_d(s) \text{ rooted at } x \text{ is visited.} & \text{otherwise} \end{cases}$$

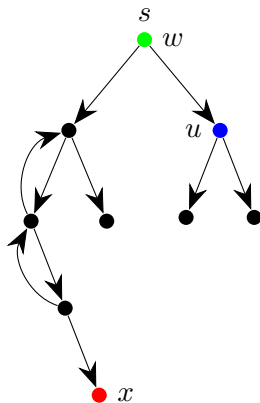




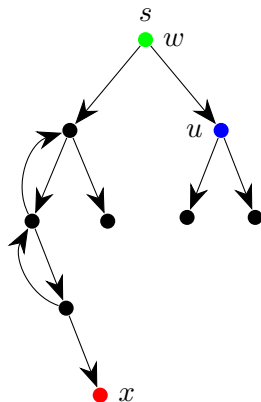


Initialize $w = s$ and $u = R(s)$

PathUncle Overview

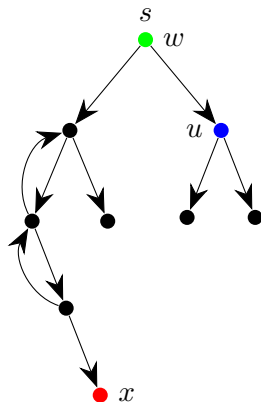


PathUncle Overview



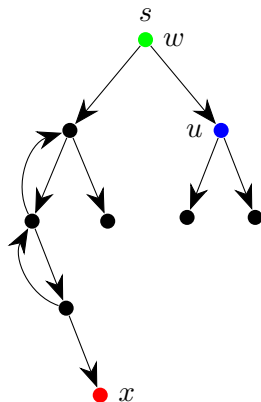
Guess $N(w)$ avoiding backedges

PathUncle Overview

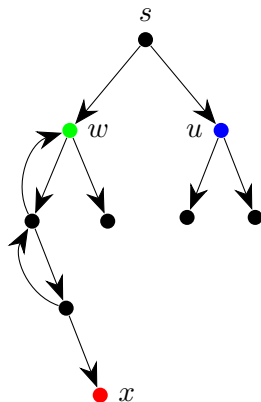


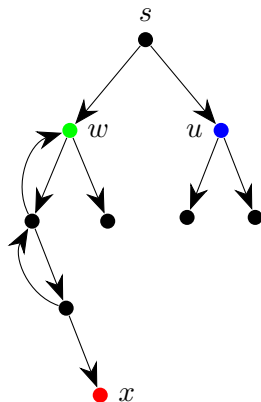
Suppose guessed left

PathUncle Overview



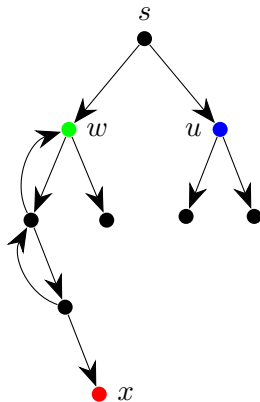
PathUncle Overview





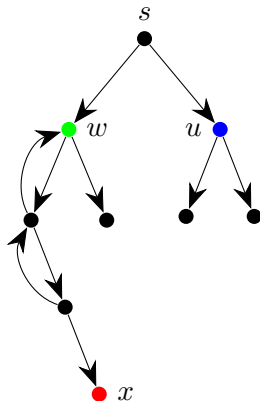
Guess $N(w)$ avoiding backedges

PathUncle Overview



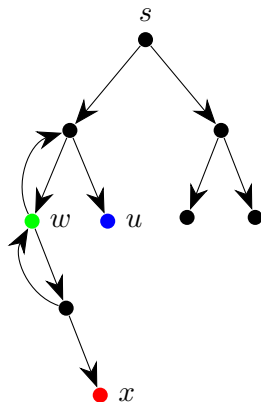
Suppose guessed left

PathUncle Overview

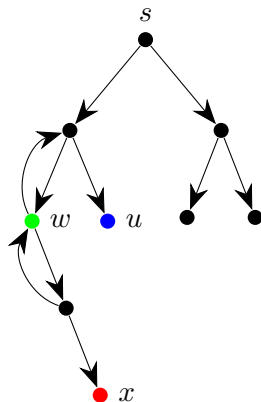


$u = R(w)$ and move left

PathUncle Overview

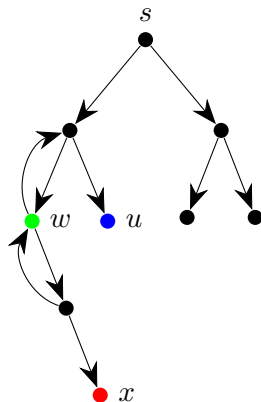


PathUncle Overview



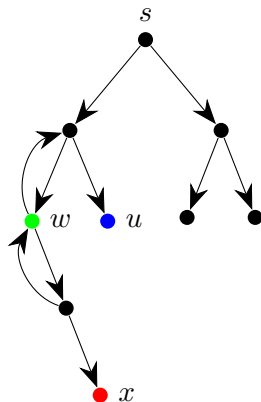
Guess $N(w)$ avoiding backedges

PathUncle Overview



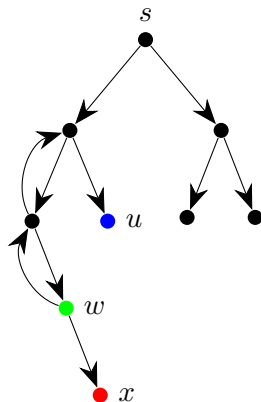
Suppose guessed right

PathUncle Overview

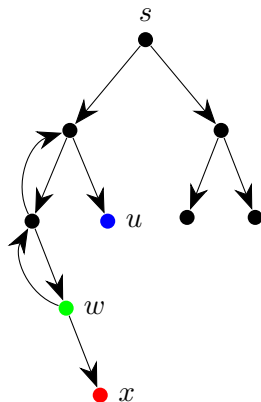


Move right

PathUncle Overview

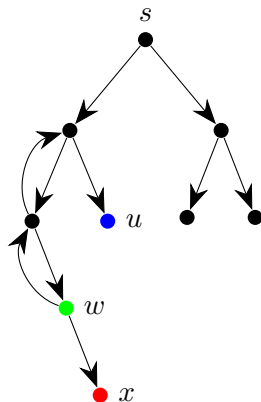


PathUncle Overview



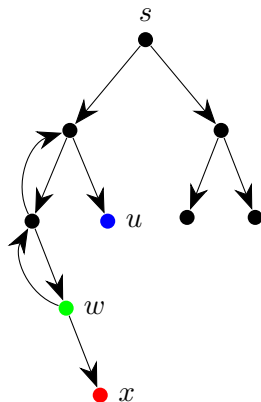
Guess $N(w)$ avoiding backedges

PathUncle Overview



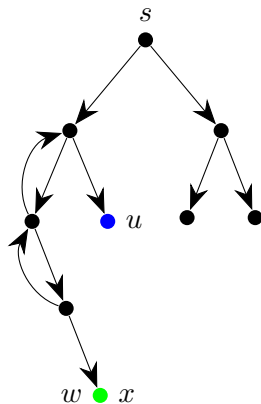
Suppose guessed right

PathUncle Overview



Move right

PathUncle Overview



Summary of results:

- Modified Lange's algorithm to work for simple path graphs in $UL \cap coUL$.

Challenges

- **Challenge 1:** Lange's algorithm rejects on back edge.
 - Modification: Detect back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.

Summary of results:

- Modified Lange's algorithm to work for simple path graphs in $UL \cap coUL$.

Challenges

- **Challenge 1:** Lange's algorithm rejects on back edge.
 - Modification: Detect back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.
- **Challenge 3:** Correctly compute pre-order successor (here, uncle) when current vertex is a leaf or both its children are back edges.

Summary of results:

- Modified Lange's algorithm to work for simple path graphs in $UL \cap coUL$.

Challenges

- **Challenge 1:** Lange's algorithm rejects on back edge.
 - Modification: Detect back edges and continue execution.
- **Challenge 2:** Return root as pre-order successor when current vertex is rightmost.
 - Modification: Detecting if current vertex is rightmost and if yes, return root.
- **Challenge 3:** Correctly compute pre-order successor (here, uncle) when current vertex is a leaf or both its children are back edges.
 - Modification: Returns uncle by guessing a path to the current vertex while storing potential uncle at each step.
- Other results
 - $ReachUL^{ReachUL} = ReachUL$

Is SimplePath-Reach $\in$ ReachUL?

Is SimplePath-Reach $\in$ ReachUL?

Consequence: Gives simultaneous $\text{poly}(n)$ time $O(\log^2 n)$ space algorithm SimplePath-Reach, improving the result of Kannan *et al.*

Questions?
Thank You!